

APUNTESWEB · INGENIERÍA INFORMÁTICA

CIBER

Tercer curso · 2.º cuatrimestre · 5 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Introducción

La **ciberseguridad** busca la **protección** de los **activos** de un sistema informático: los datos, el software, el hardware y las comunicaciones.

Los **activos** se valoran como elementos **reemplazables fácilmente**, elementos con **especial valor** para el negocio/usuario o elementos únicos e **irreemplazables**.

Proteger significa tomar las medidas adecuadas al valor de los activos

1.1 Activos

1.1.1 Los Datos

Los **datos** son información almacenada/distribuida/compartida en distintos formatos. En los equipos de **uso personal** cada día hay más datos que buscan los delincuentes. Incluso datos como el **DNI** pueden usarse para un ataque de suplantación de identidad. Los datos de una **empresa** pueden ser deseados por competidores, espías, gobiernos, etc., por lo que es fácil ser objetivo de ataques maliciosos.

Hay que valorar cuáles son las **medidas más adecuadas** para proteger los datos según su valor y lo que supondría su pérdida.

1.1.2 El software

Uno de los objetivos de los atacantes de un sistema es hacerse con el **control del software** para que trabajen para ellos. Existen dos métodos de ataque básico al software:

- Buscar **puntos débiles**: fallos del sistema operativo y vulnerabilidades conocidas de los programas instalados.
- Introducir un **caballo de Troya**: como un gestor de correo que, además de gestionar el correo, envía copia de los mensajes al atacante.

Para protegerse de este tipo de ataques, hay que tener los programas actualizados y evitar instalar programas de procedencia dudosa.

1.1.3 El hardware

Una persona con conocimientos informáticos con **acceso físico** a un equipo, aún si tener su contraseña, puede hacer muchas averiguaciones sobre él. Los **dispositivos móviles** suponen un riesgo mayor al tener más posibilidad de extravío o robo. El **IoT** (Internet of Things) incrementa la superficie de posibles ataques.

No hay que descuidar las medidas de control de acceso físico a **instalaciones**.

1.1.4 Las comunicaciones

Normalmente, el canal para transmitir información entre equipos es **Internet**, que emplea múltiples **nodos de interconexión** a los que tienen acceso **muchos usuarios**. Esto unido al hecho de que los protocolos de comunicación más empleados no **implementan mecanismos de seguridad adecuados**, provocan que cualquier persona con acceso a dichos nodos o a alguna de las redes intermedia pueda **intercepta** y leer/modificar los mensajes.

La solución a este problema es emplear **técnicas criptográficas**.

1.2 Amenazas

1.2.1 Ataques Aleatorios

Posibles **motivaciones**:

- **Diversión** de alguien que busca sistemas mal protegidos para alguna **travesura**
- Instalar en nuestro sistema **programas espía** para capturar datos nuestros y aprovecharlos en su beneficio.
- **Controlar nuestro sistema** para emplearlo como cabeza de puente en ataques contra otros sitios ocultando así su verdadera identidad. Ej: **escaneo al azar de direcciones IP y puertos abiertos**.

1.2.2 Ataques Dirigidos

Posibles **motivaciones**:

- **Competencia industrial**: interés de una empresa rival en acceder a nuestra comunicaciones internas
- **Ataques de organizaciones religiosas, políticas, ONGs** que defienden ideas contrapuestas a las de la organización.
- **Afán de notoriedad**, si somos por ejemplo una empresa con cierta imagen pública.
- **Facilidad**: ataques a sistemas porque son fáciles de atacar.

Muchas empresas suponen que los ataques siempre son externos, y por eso se dedican a establecer una defensa perimetral, pero **los ataques pueden ser internos** (provenientes de un empleado descontento o un infiltrado para hacer espionaje industrial).

Método Oportunidad-Objetivo

- **Oportunidad**: el activo es vulnerable en un determinado aspecto
- **Método**: existe un mecanismo par atacar dicha vulnerabilidad
- **Motivo**: existe un interés por parte del atacante

1.3 Servicios básicos de seguridad

- **Disponibilidad:** asegurar que la información puede ser utilizado por las partes autorizadas
- **Confidencialidad:** asegurar que la información es visto solo por partes autorizadas
- **Integridad:** asegurar que la información es modificada únicamente por partes autorizadas.
- **Autenticación:** asegurar que el origen de la información es quien dice ser
- **No repudio:** impedir que alguien niegue haber realizado una acción cuando efectivamente lo ha hecho
- **Control de acceso:** garantizar acceso únicamente a partes autorizadas a los recursos.

2. Criptografía

2.1 Criptografía simétrica

2.1.1 Idea general y conceptos básicos

La **criptografía** busca ocultar el significado de un mensaje para que solo su **destinatario** pueda interpretarlo. En un criptosistema intervienen el **mensaje en claro** m , el **mensaje cifrado** c , una **clave** k y dos operaciones: **cifrado** E y **descifrado** D .

En un **cifrado simétrico** la misma clave sirve para cifrar y descifrar:

1. $c = E_k(m)$
2. $m = D_k(c)$
3. $D_k(E_k(m)) = m$

En la criptografía clásica dominaron dos ideas. La **transposición** reordena las letras de un mensaje; un ejemplo histórico es la **escítala espartana**. La **sustitución** cambia cada letra por otra; el ejemplo más conocido es el **cifrado César**, que desplaza cada letra n posiciones en el alfabeto.

Conviene separar algunos términos:

- **Criptología**: criptografía + criptoanálisis.
- **Criptografía**: arte y ciencia de mantener mensajes seguros.
- **Criptoanálisis**: arte y ciencia de romper esos mensajes.
- **Criptosistema**: conjunto de algoritmos, claves y protocolo de uso.
- **Clave**: parámetro que inicializa y personaliza el algoritmo.

2.1.2 Criptoanálisis

El **criptoanálisis** intenta romper un cifrado o, al menos, extraer información útil. Sus objetivos típicos son descifrar un mensaje concreto, reconocer **patrones**, inferir significado por el **tamaño** o la **frecuencia** de los mensajes, deducir la **clave** o encontrar debilidades en el **algoritmo**, la **implementación** o el **entorno** de uso.

Se suele asumir que el atacante conoce el **algoritmo**, pero no la **clave**. Los modelos clásicos de ataque son:

- **Solo texto cifrado**: el atacante solo ve textos cifrados.
- **Texto plano conocido**: conoce pares de texto claro y texto cifrado.
- **Texto plano elegido**: puede elegir mensajes y obtener su cifrado.

Los ataques pueden ser **matemáticos**, si explotan propiedades del algoritmo, o **estadísticos**, si buscan correlaciones en letras, pares de letras, tripletas o mensajes cifrados de forma similar. Cuando el espacio de claves es pequeño, también es viable la **fuerza bruta**. El cifrado César, por ejemplo, solo tiene 27 claves posibles.

2.1.3 Cifrados clásicos

La **sustitución monoalfabética genérica** construye un alfabeto cifrado colocando al azar las letras del alfabeto llano. Eso da lugar a $27! \approx 10^{28}$ alfabetos posibles. Aun así, la **frecuencia de aparición** de las letras en un idioma permite atacar el sistema: si el mensaje conserva demasiados patrones, el cifrado acaba cediendo ante el análisis estadístico.

El **cifrado de Vigenère** mejora a César usando una clave de varias letras, es decir, varias sustituciones de César encadenadas. Históricamente se rompió en dos fases:

1. Se estudian las repeticiones de secuencias para estimar la **longitud de la clave**.
2. Se divide el texto en bloques y cada bloque se ataca como una **sustitución monoalfabética**.

Enigma, diseñada por **Arthur Scherbius** en 1920, mecanizó este planteamiento. Empezó como una sustitución monoalfabética, pero al girar tras cada letra pasó a comportarse como un cifrado **polialfabético**. Con 3 rotores encadenados se obtenían $26^3 = 17576$ alfabetos posibles; al añadir **reflector**, **clavijero** y rotores intercambiables, el espacio crecía hasta unos **10000 billones** de configuraciones.

El **cuaderno de uso único** (*one-time pad*) lleva la idea al límite: es un Vigenère con una clave **aleatoria**, **tan larga como el mensaje** y **no reutilizada**. En esas condiciones se considera **probablemente irrompible**. Su debilidad práctica no está en el algoritmo, sino en la **generación y distribución** de la clave. Si la clave se genera con un pseudoaleatorio débil, el sistema deja de ser seguro.

2.1.4 Primitivas y criterios de diseño

Los cifrados modernos combinan unas pocas **primitivas** básicas:

- **Sustitución**: reemplazar un conjunto de bits por otro.
- **Transposición**: cambiar el orden de los bits.
- **Confusión**: hacer compleja la relación entre texto claro, clave y texto cifrado.
- **Difusión**: repartir la información del texto claro por todo el texto cifrado.

Estas técnicas se emplean para proteger la **confidencialidad**, es decir, para que solo quien conoce la clave pueda acceder al contenido.

Un criptosistema solo merece confianza si cumple tres condiciones: está basado en **matemáticas sólidas**, ha sido **analizado públicamente** por expertos y ha resistido la **prueba del tiempo**.

2.1.5 Cifradores de bloque

Un **cifrador de bloque** divide el mensaje en bloques de n bits y cifra cada uno para producir otro bloque de n bits. Si se usa sin un modo adecuado, dos bloques iguales de texto claro producen dos bloques iguales de texto cifrado. A cambio, un buen cifrador de bloque hace que muchos bits del resultado dependan de muchos bits de la entrada, y un error de transmisión suele afectar solo al bloque donde aparece.

DES y su evolución

DES trabaja con bloques de **64 bits** y una clave total de **64 bits**, de los que **56 bits** son efectivos y **8 bits** se usan para paridad. El cifrado se realiza en **16 iteraciones** y combina **sustitución** y **permutación**; en términos de Shannon, la sustitución aporta sobre todo **confusión** y la permutación aporta **difusión**.

Con 2^{56} claves posibles, DES parecía suficiente en su momento. A **1000 claves por segundo**, romperlo por fuerza bruta llevaría más de **dos millones de años**. Ese margen desapareció con el tiempo: en **1998**, la **Electronic Frontier Foundation** construyó una máquina capaz de descifrar DES en **menos de tres días**.

Las variantes más citadas fueron:

Variante	Idea	Observación
DES	Una clave de 56 bits	Hoy es insuficiente.
Doble DES	Cifrar dos veces con dos claves	No ofrece la mejora que intuitivamente se esperaría.
Triple DES de dos claves	Esquema EDE con dos claves de 56 bits	Fuerza aproximada de unos 80 bits.
Triple DES de tres claves	Esquema EDE con tres claves de 56 bits	Fuerza aproximada de unos 112 bits.

La sustitución de DES fue gradual. El **NIST** abrió en **enero de 1997** el proceso para elegir un reemplazo, preseleccionó **15 algoritmos** en **agosto de 1998**, dejó **5 finalistas** en **abril de 1999** (**MARS**, **RC6**, **Rijndael**, **Serpent** y **Twofish**) y eligió **Rijndael** como **AES** en **octubre de 2000**.

AES y modos de uso

AES es el estándar actual de cifrado simétrico por bloques. Trabaja con bloques de **128 bits** y admite claves de **128, 192 o 256 bits**, con **10, 12 o 14 rondas** respectivamente. Cada ronda combina operaciones como **SubBytes**, **ShiftRows**, **MixColumns** y **AddRoundKey**.

El algoritmo por sí solo no basta: importa mucho el **modo de uso**.

ECB (*Electronic Code Book*) cifra cada bloque de forma **independiente**. Es sencillo, pero tiene dos problemas graves: las **repeticiones** del texto claro se reflejan en el texto cifrado y, si un atacante

reconoce bloques, puede intentar **recombinarlos** para fabricar mensajes válidos.

CBC (*Cipher Block Chaining*) evita eso encadenando los bloques. Antes de cifrar cada bloque, se hace un **XOR** con el bloque cifrado anterior. El primer bloque necesita un **vector de inicialización (IV)**.

CTR usa un **contador** y un valor inicial aleatorio para convertir un cifrador de bloque en un esquema de tipo **flujo**. Su ventaja principal es la **paralelización**.

GCM añade además **autenticación**. El **NIST** lo recomienda para cifrado autenticado y, combinado con **AES**, aparece en protocolos como **TLS**. En esencia, AES se aplica sobre el **IV/contador** y el resultado se combina con el texto claro; además se calcula una etiqueta de autenticación. Aun así, un mal uso del **IV/nonce** puede introducir vulnerabilidades graves.

2.1.6 Cifradores de flujo

Un **cifrador de flujo** procesa el mensaje bit a bit o byte a byte. Su funcionamiento básico es:

1. A partir de una **semilla** se genera una secuencia pseudoaleatoria del mismo tamaño que el mensaje.
2. Esa secuencia se combina con el texto claro, normalmente mediante **XOR**.
3. Emisor y receptor comparten la semilla para poder reproducir la misma secuencia.

Para que la secuencia cifrante sea segura debe tener un **período** al menos tan largo como el mensaje y una **distribución** de bits y de **rachas** compatible con una secuencia pseudoaleatoria. En la práctica, semillas de entre **120 y 250 bits** permiten obtener períodos enormes.

La propuesta clásica es el **cifrador de Vernam** (1917): se parte de una secuencia binaria aleatoria S , derivada de una clave secreta compartida K , y se cifra con **XOR**. Si la secuencia es realmente aleatoria y no se reutiliza, el esquema se acerca al cuaderno de uso único.

Un generador pseudoaleatorio sencillo puede implementarse con un **registro de desplazamiento**: el registro empieza cargado con la clave, se desplaza un bit en cada iteración, se toma un bit como salida y se calcula el nuevo bit con una función $f(B_n, B_{n-1}, \dots, B_1)$. Esa función debe evitar repeticiones cortas, no ser predecible y ser fácil de implementar.

Dos algoritmos históricos ilustran bien los problemas prácticos:

- **RC4** usa claves de longitud variable. Fue **propietario** hasta que se filtró en **septiembre de 1994**, llegó a emplearse en **SSL, WEP y WPA** y era unas **diez veces más rápido** que DES. Sus debilidades de implementación acabaron permitiendo ataques reales.
- **A5**, propuesto en **1994** y no publicado en su diseño original, se usó en el cifrado del enlace **GSM**. Sus debilidades mostraron una lección clásica: en criptografía, ocultar el algoritmo en lugar de someterlo a revisión pública suele acabar mal.

2.1.7 Limitaciones de la criptografía simétrica

El gran problema es la **distribución de claves**. Si hay n usuarios y cada pareja necesita una clave distinta, hacen falta

$$\frac{n(n-1)}{2}$$

claves compartidas, además de un **canal seguro** para repartirlas. A eso se suma el riesgo de **fuerza bruta**, que obliga a usar claves suficientemente largas y a **renovarlas** con cierta frecuencia.

2.2 Criptografía de clave pública

2.2.1 Intercambio de claves: Diffie-Hellman

La criptografía de clave pública nace, en buena parte, para aliviar el problema de distribución de claves. **Diffie-Hellman** (1976) permite que dos partes acuerden una **clave simétrica común** sobre un canal inseguro. Se apoya en la dificultad del **logaritmo discreto**: dado un primo grande p , una base a y un valor $n = a^k \bmod p$, recuperar k es computacionalmente inviable cuando p es suficientemente grande.

El intercambio clásico funciona así:

1. Alice y Bob acuerdan públicamente p y a . Ejemplo: $p = 11$ y $a = 7$.
2. Alice elige un secreto $x = 3$ y envía $A = a^x \bmod p = 2$.
3. Bob elige un secreto $y = 6$ y envía $B = a^y \bmod p = 4$.
4. Alice calcula $K = B^x \bmod p = 9$.
5. Bob calcula $K = A^y \bmod p = 9$.

Ambos obtienen la misma clave K sin haberla transmitido directamente.

Por sí solo, Diffie-Hellman **no autentica** a las partes, así que es vulnerable a ataques **man-in-the-middle**. Por eso necesita mecanismos adicionales de autenticación. Se ha usado en **SSL/TLS**, **SSH**, **Secure FTP** e **IPsec**.

2.2.2 Idea de la clave pública

En criptografía de clave pública cada usuario tiene dos claves:

- una **clave privada** k_S , que solo conoce su propietario;
- una **clave pública** k_P , que puede conocer cualquiera.

Ambas son inversas en cierto sentido: lo que una hace, la otra lo deshace.

Su funcionamiento básico para **confidencialidad** es:

1. $c = E_{k_P}(m)$
2. $m = D_{k_S}(c)$
3. $D_{k_S}(E_{k_P}(m)) = m$

Un sistema de clave pública debe permitir cifrar y descifrar con facilidad usando la clave correcta, pero hacer **computacionalmente inviable** recuperar la **clave privada** a partir de la **pública** o romper el sistema mediante ataques de **texto claro elegido**.

En cuanto a servicios de seguridad, la idea general es simple: la **clave pública** sirve para proteger la **confidencialidad** y la **clave privada** sirve para demostrar **autoría**. En la práctica, esto último se implementa con **firmas digitales** sobre un **resumen**, no cifrando el mensaje completo con la clave privada.

2.2.3 RSA

RSA se basa en la dificultad de **factorizar** números grandes y de calcular ciertas **raíces discretas** módulo un número grande. Dos números son **primos relativos** si no comparten factores. La función $\phi(n)$ cuenta cuántos enteros positivos menores que n son primos relativos con n ; por ejemplo, $\phi(10) = 4$ porque 1, 3, 7 y 9 son coprimos con 10.

La generación de claves se resume así:

1. Se eligen dos primos grandes p y q .
2. Se calcula $n = p \cdot q$ y $\phi(n) = (p - 1)(q - 1)$.
3. Se elige $e < n$ tal que $\gcd(e, \phi(n)) = 1$.
4. Se calcula d de forma que $e \cdot d \equiv 1 \pmod{\phi(n)}$.

La **clave pública** es $k_P = (e, n)$ y la **clave privada** es $k_S = d$.

$$c = m^e \bmod n$$

$$m = c^d \bmod n$$

RSA tiene varias ventajas frente al cifrado simétrico puro: no necesita un **canal seguro** para repartir claves y basta con un **par de claves por usuario**. Sus pegas también son claras: necesita **claves grandes** (al menos **2048 bits**), es **más lento** que el cifrado simétrico, tiene límites prácticos sobre el tamaño del mensaje y depende de una **infraestructura de confianza** para asociar identidades y claves públicas.

Desde el punto de vista de seguridad, RSA puede soportar **confidencialidad**, **autenticación**, **integridad** y **no repudio**. Para evitar ataques estadísticos o manipulaciones obvias, el mensaje no debe tratarse como texto crudo: se trabaja en **bloques grandes** y con **padding** adecuado. Aun así, no se usa para cifrar grandes volúmenes de datos: en la práctica se emplea sobre todo en esquemas **híbridos** y para **encapsular claves**.

2.2.4 Criptografía híbrida

La **criptografía híbrida** combina lo mejor de ambos mundos: la criptografía de **clave pública** se usa para **autenticar** y para **intercambiar** una **clave de sesión simétrica**, y la criptografía **simétrica** se usa después para cifrar los datos, mucho más rápido.

En **TLS**, por ejemplo, pueden aparecer certificados **RSA** para autenticación, mecanismos **Diffie-Hellman** o **ECDHE** para acordar la clave de sesión y algoritmos simétricos como **AES-GCM** para proteger el tráfico.

La comparación práctica entre opciones habituales es:

Opción	Tamaño de clave equivalente	Impacto en rendimiento	Forward secrecy	Uso típico
RSA	2048 bits	Alto	No	Sistemas heredados
DHE	2048 bits	Moderado	Sí	Algunas aplicaciones
ECDHE	256 bits ECC	Bajo	Sí	Opción habitual en TLS 1.3

2.2.5 Evolución: curvas elípticas y presión poscuántica

El crecimiento de la capacidad de cálculo y la posibilidad teórica de que una computación cuántica escalable ejecute el **algoritmo de Shor** presionan a sistemas como **RSA** o **Diffie-Hellman**. Una respuesta inmediata fue **aumentar el tamaño de las claves RSA**, pero eso empeora la **eficiencia**.

Las **curvas elípticas** ofrecen la misma seguridad con claves mucho más pequeñas: una clave de **256 bits ECC** suele compararse con una clave **RSA de 3072 bits**. Eso reduce coste computacional y ancho de banda.

2.2.6 Intercambio de claves con curva elíptica

En **ECDH** dos partes acuerdan una clave compartida sobre una **curva elíptica pública** y un **punto generador G** .

Si Alice tiene clave privada a y Bob clave privada b :

1. Alice calcula su clave pública $A = aG$.
2. Bob calcula su clave pública $B = bG$.
3. Alice obtiene el secreto compartido como $S = aB$.
4. Bob obtiene el mismo secreto como $S = bA$.

Como $a(bG) = b(aG)$, ambos llegan al mismo punto secreto. A partir de él se **deriva** una clave simétrica, por ejemplo para **AES**.

La seguridad depende de la dificultad del **logaritmo discreto en curvas elípticas**. Aun así, ECDH **no cifra ni autentica** por sí solo: necesita integrarse con otros mecanismos y conviene usar **curvas estandarizadas y auditadas**.

2.3 Funciones hash y firma digital

Las **funciones hash** o de **resumen** se usan para comprobar si un bloque de datos ha sido **modificado**. No ocultan el contenido como un cifrado: lo transforman en una huella corta y de longitud fija.

2.3.1 Funciones hash criptográficas

Una función hash criptográfica $h : A \rightarrow B$ debe cumplir varias propiedades:

- ser **fácil** de calcular para cualquier entrada;
- ser **muy difícil de invertir**;
- hacer **muy difícil** encontrar dos entradas distintas con el mismo resumen;
- producir un resumen **corto** y de **longitud fija**;
- comportarse como una función de **una sola vía**.

Sus usos más habituales son el **chequeo de integridad**, la **autenticación**, los protocolos de **comunicación**, la **firma digital**, algunos mecanismos de **cifrado** y el **almacenamiento de contraseñas**. En este último caso conviene añadir una **sal**: si dos usuarios tienen la misma contraseña y no se usa sal, acabarán teniendo el mismo hash.

Colisiones

Si $x \neq x'$ y $h(x) = h(x')$, existe una **colisión**. Que las colisiones existan es inevitable por el **principio del casillero**: si hay más entradas posibles que salidas posibles, alguna salida corresponderá a varias entradas. Lo importante no es eliminarlas, sino hacer **computacionalmente inviable encontrarlas**.

Con clave y sin clave

Los resúmenes pueden ser:

- **Con clave**: incorporan una clave compartida. El ejemplo clásico es un **MAC**; en estos apuntes se cita **DES en modo de encadenamiento** como aproximación histórica.
- **Sin clave**: no usan clave criptográfica. Aquí entran **MD5**, **SHA-1**, **SHA-2** y **SHA-3**.

Las familias más importantes son:

Familia	Bloques de entrada	Tamaño del resumen	Situación
MD5	512 bits	128 bits	En desuso por debilidades.

Familia	Bloques de entrada	Tamaño del resumen	Situación
SHA-1	512 bits	160 bits	Ya no se considera seguro para varios usos desde 2017.
SHA-2	512 o 1024 bits	224, 256, 384 o 512 bits	Familia más usada actualmente.
SHA-3	Construcción distinta a SHA-2	224, 256, 384 o 512 bits	Alternativa moderna.

En términos prácticos, los tamaños más recomendables dentro de estas familias suelen ser **SHA-256**, **SHA-384**, **SHA-512**, **SHA3-256** y **SHA3-512**.

2.3.2 Firma digital

La **firma digital** no busca dar **confidencialidad**; su función real es garantizar **autenticidad**, **integridad** y **no repudio**. Se basa en criptografía de **clave pública**:

1. El firmante calcula un **resumen** del documento.
2. Firma ese resumen con su **clave privada**.
3. Cualquier tercero usa la **clave pública** correspondiente para verificar la firma y comparar el resumen.

Una firma digital debe ser:

- **infalsificable**: nadie debería poder firmar sin la clave privada;
- **auténtica**: el receptor debe poder atribuirle al firmante correcto;
- **no alterable**: cualquier manipulación debe resultar evidente;
- **no reutilizable**: no debe poder trasplantarse a otro documento sin que se note.

Cuando la verificación es correcta, el receptor obtiene tres garantías: el remitente es quien dice ser (**autenticación**), el mensaje no cambió (**integridad**) y el remitente no puede negar razonablemente su intervención (**no repudio**).

2.4 Infraestructura de clave pública (PKI)

2.4.1 El problema de la confianza

En el mundo físico confiamos en documentos como el **DNI** o el **pasaporte** porque los emite una **autoridad** reconocida. En el mundo digital ocurre algo parecido: una firma con clave privada demuestra intervención técnica, pero sigue haciendo falta un mecanismo que diga **de quién es** realmente la clave pública asociada.

Ahí entran los **certificados** y las **autoridades certificadoras**.

2.4.2 Certificados digitales

Un **certificado digital** contiene los **datos públicos** de un usuario firmados por una **CA** (*Certification Authority*). Como mínimo incluye:

- la **identidad** del titular;
- su **clave pública**;
- la **fecha de emisión** y el período de validez;
- información adicional relevante.

Ejemplo clásico: Bill obtiene de Carlos el certificado de Amy. Si Bill ya conoce la **clave pública de Carlos**, puede verificar la firma de Carlos y, por tanto, confiar en que la clave pública incluida realmente pertenece a Amy. El problema no desaparece: simplemente se desplaza un nivel arriba, hasta la **CA**.

X.509 v3

Los certificados más habituales siguen el estándar **X.509**. En las versiones antiguas se usaban nombres de estilo **X.500**, con campos como **C** (*Country*), **O** (*Organization*), **OU** (*Organizational Unit*) y **CN** (*Common Name*). Un ejemplo sería:

```
C=ES, O=USC, OU=ETSE, OU=DEC, CN=Puri Cariñena
```

La **versión 3** amplía esto y permite identificar al sujeto mediante nombre X.500, **correo electrónico**, **dominio**, **URL**, **dirección IP** u otros nombres registrados, incluso varios a la vez.

Entre los campos más importantes de X.509v3 están la **versión**, el **número de serie**, el **algoritmo de firma/hash**, el **emisor**, el **período de validez**, el **sujeto**, la **clave pública** y la **firma** de la CA.

2.4.3 Componentes y operaciones de una PKI

La **PKI** organiza claves públicas, claves privadas, certificados y autoridades certificadoras de forma **gestionable**, **flexible** y **segura**. Su objetivo es asociar correctamente una **identidad** con una **clave pública**. La criptografía simétrica no sirve para esto porque la clave es **compartida** por las partes. Si la asociación identidad-clave es errónea, desaparece la privacidad entre ellas.

Los componentes más habituales son:

- **Usuario**: puede ser una persona, una máquina o un programa.
- **Autoridad certificadora (CA)**: emite certificados.
- **Autoridad de registro (RA)**: registra usuarios y, opcionalmente, genera claves o verifica la **prueba de posesión** de la clave privada.
- **Repositorio de certificados**: publica certificados y CRLs.
- **Autoridad de validación (VA)**: valida certificados en línea.
- **Autoridad de sellado de tiempo (TSA)**: aporta fecha y hora confiables.

- **Hardware y tarjetas criptográficas:** protegen claves y operaciones sensibles.

Las acciones más habituales en una PKI son el **registro** de usuarios, con distintos niveles de autenticación; la generación y almacenamiento del **par de claves**; la posible **recuperación de claves**; la **revocación** de certificados y la **certificación cruzada** entre autoridades. La recuperación de claves puede ser necesaria, por ejemplo, si un empleado abandona la organización y destruye su clave privada o si existe un **mandato judicial**.

2.4.4 Emisión, validación y revocación

Emisión de un certificado

Un proceso típico de emisión es:

1. El usuario genera un par k_P / k_S en una **tarjeta** o en su equipo.
2. La **clave privada** se protege con un **PIN** y no debería salir nunca del soporte seguro.
3. La **clave pública** se envía a la CA en una solicitud, por ejemplo **PKCS#10**.
4. La **RA**, si existe, verifica la identidad y aprueba la solicitud.
5. La **CA** firma los datos y devuelve el certificado, por ejemplo en formato **PKCS#7**.

Validación de un certificado X.509

Validar un certificado implica, como mínimo:

1. Comprobar que el **nombre** del certificado corresponde al uso esperado.
2. Obtener la **clave pública del emisor** correcta.
3. Verificar la **firma** del certificado.
4. Recalcular y comparar el **resumen**.
5. Revisar el **período de validez**.
6. Comprobar que el certificado **no ha sido revocado**.

Revocación

Un certificado puede revocarse porque el usuario pierde el **PIN**, la **tarjeta**, sospecha compromiso de su clave privada o porque la propia **CA** decide retirar la confianza.

Para gestionar esto, las CAs publican periódicamente **CRLs** (*Certificate Revocation Lists*), que indican qué certificados ya no son válidos. Una CRL identifica la lista, su período de validez, la CA emisora y los **números de serie** de los certificados revocados. Los navegadores suelen consultar el estado mediante **OCSP** (*Online Certificate Status Protocol*).

Formatos habituales

Los formatos más comunes son **BER**, **DER**, **CER**, **PER**, **PEM** y **PKCS#12**. **DER** y **PEM** son especialmente habituales en herramientas como **OpenSSL**. **PKCS#12** es importante porque puede

incluir la **clave privada** y la **cadena de certificados** hasta la raíz de la CA; suele usarse para certificados personales en navegadores y acostumbra a llevar extensión **.p12**.

2.4.5 Certificación cruzada y cadenas de confianza

Cuando existen muchas CAs aparece un nuevo problema: una parte puede no confiar directamente en la CA de la otra. La **certificación cruzada** resuelve esto haciendo que dos CAs se firmen mutuamente.

En el ejemplo clásico:

- Amy tiene un certificado firmado por **Carlos**.
- Bill tiene un certificado firmado por **Dolores**.
- Carlos emite un certificado para Dolores.
- Dolores emite un certificado para Carlos.

Así, si Amy confía en Carlos, puede validar el certificado de Dolores y, a través de él, el de Bill.

A veces no se dispone del certificado de la CA exacta que firmó un certificado. Por eso existen las **cadenas de certificación**: una CA intermedia puede estar firmada por otra CA, y así sucesivamente hasta llegar a una **raíz** ya conocida por el sistema o el navegador.

Un ejemplo práctico es este: el navegador ya confía en la CA de **Telefónica**; recibimos un documento firmado por un empleado de **Banesto**; si además conseguimos un certificado de la **CA de Banesto** firmado por la **CA de Telefónica**, podemos verificar primero la CA de Banesto y después el certificado del empleado.

2.4.6 Tipos de CA y transición a ECC

Las CAs pueden adoptar distintas formas:

- **Organizaciones** que emiten certificados para sus propios miembros.
- **Empresas comerciales** de prestigio cuyos certificados raíz suelen venir preinstalados en navegadores.
- **Organizaciones sin ánimo de lucro** que ofrecen certificados gratuitos, por ejemplo para **SSL/TLS**.
- **Entidades públicas** que emiten certificados para la ciudadanía. En España destacan **CERES** (FNMT-RCM) y los certificados del **DNI electrónico** emitidos por la Dirección General de la Policía.

Como tendencia reciente, muchas infraestructuras están migrando de **RSA** a **curvas elípticas**. En la imagen original de estos apuntes se cita, por ejemplo, el plan anunciado por la **FNMT-RCM** el **9 de diciembre de 2025** para transicionar a certificados **ECC**, manteniendo el uso de claves **RSA de 2048 bits** hasta el **31 de diciembre de 2026** y limitando la validez de certificados emitidos con ese algoritmo hasta el **31 de diciembre de 2028**.

3. Seguridad en programas y aplicaciones

3.1 Programación segura

El **CERT** (*Computer Emergency Response Team*) actúa como centro de coordinación de avisos de seguridad a nivel global. En la práctica, junto con otros **CSIRT**, pone de relieve una idea importante: buena parte de los riesgos actuales vienen de **aplicaciones** y **servicios en Internet**, y muchos de esos problemas **no se arreglan con criptografía**.

La seguridad no es algo que se añada al final del desarrollo. Depende del **entorno real** donde se ejecutará el software, de la **política de seguridad** definida y de las decisiones tomadas desde el diseño. Los atacantes no “crean” por arte de magia los agujeros: normalmente **explotan fallos que ya existen**.

3.1.1 Introducción

Es habitual que las aplicaciones salgan al mercado con errores que se corrigen después, a medida que los atacantes los van encontrando. Eso tiene varias consecuencias: los **parches** solo solucionan los fallos ya detectados, muchos administradores **tardan meses en aplicarlos** o no los aplican nunca, y económicamente suele ser más rentable **detectar y eliminar** esos problemas **antes** del lanzamiento.

El patrón típico es conocido: cuando una vulnerabilidad se **descubre** y se **publica**, los ataques aumentan; cuando sale el **parche**, los ataques pueden seguir creciendo porque muchos sistemas siguen sin actualizarse; y el descenso real puede retrasarse incluso **más de un año**.

Metas de un proyecto software

Todo proyecto software intenta equilibrar varios objetivos, y la seguridad compite con algunos de ellos:

Meta	Idea principal	Relación con la seguridad
Funcionalidad	Que el programa haga lo que debe hacer	Suele verse como el objetivo principal
Usabilidad	Que sea fácil de usar	Una mala interacción favorece errores humanos; además, puede chocar con medidas de seguridad
Eficiencia	Que consuma poco tiempo y pocos recursos	Algunas defensas añaden sobrecarga
<i>Time-to-market</i>	Llegar rápido al mercado	Las prisas suelen perjudicar a la seguridad

También conviene separar dos tipos de prueba. Un **test de funcionalidad** verifica el comportamiento esperado en situaciones normales. Un **test de seguridad** comprueba que la aplicación siga comportándose correctamente en situaciones anómalas: entradas demasiado largas, parámetros erróneos, intentos de conexión inválidos o uso malicioso deliberado.

3.1.2 Principios básicos

La programación segura suele resumirse en diez reglas prácticas:

1. **Asegurar los puntos más débiles.** Un sistema es tan fuerte como su eslabón más débil. Si una comunicación va cifrada entre A y B, puede ser más eficaz atacar A o B; si existe un cortafuegos, quizá sea mejor atacar la aplicación expuesta. A veces el punto más débil ni siquiera está en el software, sino en el entorno o en la **ingeniería social**.
2. **Practicar la defensa en profundidad.** Hay que superponer capas para que el fallo de una no suponga la caída total del sistema. En una base de datos, por ejemplo, pueden coexistir cortafuegos corporativo, cifrado de comunicaciones internas, cortafuegos local, protección física y cifrado de los propios datos.
3. **Fallar de forma segura.** Los fallos son inevitables; lo importante es que, si una acción falla, el sistema quede **tan seguro como antes** de iniciarla. De hecho, forzar fallos es una técnica clásica de ataque.
4. **Aplicar privilegios mínimos.** Cada componente debería tener solo los permisos imprescindibles y durante el menor tiempo posible. El caso típico es un servicio que necesita privilegios elevados para una operación inicial y que después debería **soltarlos**. Muchos productos comerciales no lo hacen por comodidad. El ejemplo clásico es un servidor de correo en UNIX: puede necesitar privilegios para abrir el puerto 25, pero no para conservarlos durante toda su ejecución; sin embargo, programas como `sendmail` los mantuvieron históricamente más tiempo del necesario.
5. **Compartimentalizar.** Conviene separar subsistemas según sus privilegios y según la parte del sistema con la que necesitan interactuar. Cuanto más aisladas estén las piezas críticas, más difícil será una escalada lateral.
6. **Apostar por lo simple.** El software complejo se entiende peor y, por tanto, falla más. Esto favorece la reutilización de componentes de calidad, el uso de puntos de control claros para operaciones peligrosas, opciones seguras por defecto y un diseño que no dependa de diálogos que el usuario nunca leerá.
7. **Favorecer la privacidad.** La aplicación debe proteger los datos que le entrega el usuario. Además, exponer información innecesaria del propio programa, como su versión exacta, puede facilitar ataques.
8. **Recordar que mantener secretos es difícil.** Ocultar el diseño o la implementación rara vez funciona, sobre todo si el atacante puede estudiar el binario o la comunicación interna. La seguridad no debería depender de esa oscuridad, salvo en secretos reales como contraseñas o claves.
9. **Ser cauto con la confianza.** Un servidor no debe confiar ciegamente en el cliente ni el cliente en el servidor. Tampoco conviene asumir que terceros siguen buenas prácticas o que cualquier “producto de seguridad” es benigno. La confianza es transitiva y hay que tratarla como tal.

10. **Aprovechar los recursos de la comunidad.** Los componentes ampliamente usados, revisados durante años y analizados públicamente suelen ofrecer más confianza que las soluciones ad hoc poco examinadas.

3.2 Vulnerabilidades en el desarrollo de aplicaciones

Una **vulnerabilidad software** es un fallo en las **políticas, procedimientos o controles** de un programa que permite a alguien violar su política de seguridad. A quien aprovecha ese fallo se le llama **atacante**, y al proceso de aprovecharlo se le llama **explotación**. Un **exploit** es el programa o técnica concreta que materializa ese aprovechamiento.

3.2.1 TOCTOU

TOCTOU (*Time Of Check, Time Of Use*) es una vulnerabilidad de sincronización en la que una decisión de acceso se toma en un instante y el uso real del recurso ocurre un poco después. Ese hueco es la **ventana de vulnerabilidad**: durante ese tiempo, un atacante puede cambiar la situación y provocar un acceso incorrecto.

En accesos a ficheros suelen darse tres condiciones para que el ataque sea viable: el atacante tiene acceso a la **máquina local**, el programa vulnerable se ejecuta con **privilegios elevados** como `setuid` o `setgid`, y esos privilegios se mantienen durante la ventana de vulnerabilidad.

Las medidas típicas son conocidas. Si es posible, conviene trabajar con **descriptores o punteros de fichero** en lugar de con **nombres**, porque así el objeto abierto ya no cambia por el camino. Si no es posible, el fichero debería residir en un directorio accesible solo para el **UID del programa**. Y, en general, **no** debería usarse `setuid` o `setgid` salvo que sea estrictamente necesario.

Ejemplo: `xterm`

`xterm` actuaba como emulador de terminal para X11 y, en ciertos sistemas UNIX, necesitaba ejecutarse con privilegios elevados. Su mecanismo de registro permitía al usuario guardar entradas y salidas en un fichero: si el fichero no existía, `xterm` lo creaba; si ya existía, comprobaba que el usuario pudiera escribir en él y luego añadía el registro.

El problema es que **root puede escribir en cualquier fichero**, así que la comprobación tendía a salir bien. Entre `access` y `open` quedaba una ventana en la que un atacante podía sustituir el nombre por un **enlace simbólico** hacia un fichero sensible y lograr que el programa lo abriera con privilegios.

Para elevar la probabilidad de éxito, el atacante puede automatizar el reemplazo del enlace hasta acertar con la ventana vulnerable.

3.2.2 Desbordamiento de memoria

Un **buffer** es una zona de memoria de tamaño prefijado destinada a almacenar datos del mismo tipo. Hay **desbordamiento** cuando un programa intenta escribir más datos de los que caben. Lo que sobra pisa memoria adyacente, corrompe datos válidos y puede incluso alterar el **flujo de ejecución**.

Hay ataques **remotos**, cuando la entrada llega desde la red, y ataques **locales**, cuando un usuario con acceso al equipo intenta escalar privilegios explotando una aplicación que corre con otro usuario, por ejemplo **root**.

El impacto real depende de cuatro factores: cuánta información se escribe fuera de límites, **qué datos** se sobrescriben, si el programa vuelve a **leer** esos datos y **con qué valores** han sido sustituidos.

Por qué es un problema de seguridad

La variante clásica es el **stack-smashing**: el atacante introduce código propio en la pila y fuerza un salto a esa zona modificando la **dirección de retorno**. Desbordar el **heap** suele ser más difícil, por eso muchos desarrolladores se sienten más cómodos con reserva dinámica, pero eso no elimina el problema.

Si el programa vulnerable corre con `setuid`, el fallo puede convertirse en una vía de **escalada de privilegios**. Y si el servicio está expuesto a la red, el ataque puede llegar a ser **remoto**.

En este contexto aparecen dos términos frecuentes:

- **Shellcode**: código pensado para ejecutarse dentro del programa vulnerable y tomar su control. El ejemplo clásico abre una **shell** mediante `execve("/bin/sh", ...)`.
- **Exploit**: programa o secuencia de entrada que aprovecha una o varias vulnerabilidades para ejecutar ese código o alterar el flujo del programa.

Un exploit de pila suele combinar varias piezas: relleno para alcanzar la zona sensible, a veces una **NOP sled** para no tener que acertar una dirección exacta, el **shellcode** y una o varias **direcciones de retorno** que redirigen la ejecución.

Contramedidas

No existe una defensa única. Lo razonable es acumular barreras en varias capas:

Capa	Medidas habituales	Limitaciones
Desarrollador	Comprobar límites, usar funciones como <code>strncpy</code> , revisar con cuidado bucles con <code>getc</code> y similares, o usar lenguajes con comprobación de límites como Java o C#	Requiere disciplina y no corrige errores ya escritos
Compilador	Compiladores con comprobación de límites, <code>StackGuard</code> , protecciones de pila y avisos de gcc	Puede penalizar rendimiento y no cubre todos los desbordamientos,

Capa	Medidas habituales	Limitaciones
	frente a modificaciones sospechosas	especialmente en heap
Sistema operativo	Pila no ejecutable, ASLR	Se puede intentar rodear mediante prueba y error u otras técnicas
Red	DPI para detectar <i>shellcodes</i> , secuencias largas de NOPs u otros patrones	El malware puede ofuscarse o automodificarse

La experiencia práctica es clara: la base del **CERT** contiene muchas vulnerabilidades de este tipo, siguen apareciendo constantemente, suelen ser **fáciles de explotar** cuando existen y **no** hay una solución universal que proteja cualquier programa vulnerable.

3.3 Vulnerabilidades web

3.3.1 Ataques centrados en la web y en la sesión del usuario

Man in the Browser introduce malware en el navegador para **interceptar o modificar** lo que el usuario escribe o envía, incluso si luego la comunicación viaja cifrada.

Registro de pulsaciones o keylogging consiste en capturar todo lo que el usuario teclea para robar credenciales u otra información. Puede hacerse con malware o incluso con hardware, y no se limita al navegador.

Page in the Middle redirige al usuario a una página distinta de la que cree visitar para capturar o manipular su entrada.

Browser in the Browser simula dentro de la propia web una ventana de inicio de sesión aparentemente legítima para engañar al usuario.

Clickjacking oculta la acción real asociada a un clic. El usuario cree pulsar algo inocente, pero en realidad activa otra operación, como conceder acceso a cámara o micrófono.

3.3.2 Engaño al usuario

Phishing es cualquier mensaje o comunicación que engaña a la víctima para que entregue datos privados o haga algo inseguro.

Un **sitio web falso** imita a uno legítimo para ganarse la confianza del usuario y robar credenciales o datos sensibles.

La **sustitución en la descarga de programas** ofrece software aparentemente normal que, al descargarse o instalarse, introduce malware o spyware.

Las **herramientas falsas** se presentan como utilidades útiles o incluso como soluciones de seguridad, pero en realidad manipulan al usuario, muestran alertas falsas o instalan malware.

User in the Middle convierte al propio usuario en intermediario involuntario. Un ejemplo típico es hacerle resolver un **CAPTCHA** o completar una acción que en realidad beneficia al atacante, como la creación masiva de cuentas.

3.3.3 Mediación incompleta

La **mediación incompleta** aparece cuando el servidor confía demasiado en datos que llegan del cliente y no los vuelve a validar. Entonces basta alterar parámetros de la URL o del formulario, como **precio**, **cantidad** o **identificadores**, para cambiar el resultado.

La regla aquí es simple: **no se debe confiar en el navegador ni en la entrada del usuario**, porque ambos pueden ser manipulados antes de llegar al servidor.

3.3.4 Ataques de inyección

Un ataque de **inyección** introduce datos maliciosos para que la aplicación ejecute o interprete algo distinto de lo esperado.

Los casos más conocidos son:

- **XSS** (*Cross-Site Scripting*): se inyecta script o HTML para que el navegador ejecute contenido malicioso dentro de una web.
- **Path traversal**: se manipulan rutas como `../` para acceder a ficheros del servidor que no deberían exponerse.
- **Inyección SQL**: se introducen fragmentos SQL para alterar las consultas a la base de datos.

Las contramedidas básicas son siempre las mismas: **filtrar y sanear** toda la entrada, teniendo en cuenta cualquier codificación válida; **no hacer suposiciones** sobre lo que enviará el usuario; y apoyarse en controles del lado servidor, como **procedimientos almacenados** o mecanismos equivalentes.

3.3.5 OWASP

OWASP es una comunidad y proyecto abierto orientado a mejorar la seguridad de las aplicaciones web y a ayudar a desarrollar, comprar y mantener aplicaciones de confianza.

Uno de sus recursos más conocidos es el **OWASP Top 10**, una lista de riesgos que se actualiza con el tiempo. Ya en la transición de **2013** a **2017** cambiaron varias categorías, por ejemplo con la aparición de **XXE** y de **Insecure Deserialization** y con el peso creciente del **Broken Access Control**. La imagen usada en los apuntes muestra además la transición de **2017** a **2021**, donde aparecen categorías como **Broken Access Control**, **Cryptographic Failures**, **Injection**, **Insecure Design**, **Security**

3.4 Código malicioso

El **código malicioso** o **malware** es software que se ejecuta sin el conocimiento o autorización del propietario del equipo y realiza acciones perjudiciales para el usuario o para el sistema. A menudo se apoya en **operaciones legales**: un usuario autorizado podría realizarlas sin violar la política de seguridad, y el malware simplemente **imita** a ese usuario.

3.4.1 Tipos de malware

Los tipos más habituales son:

Tipo	Qué hace	Rasgo característico
Virus	Se replica asociado a un programa huésped e infecta otras copias	Suele activarse al ejecutar el código infectado y busca pasar desapercibido
Troyano	Parece inofensivo, pero ejecuta acciones maliciosas	Combina una función visible con otra encubierta
Rootkit	Intercepta funciones del sistema para ocultarse a sí mismo o a otro malware	Puede ocultar procesos, ficheros, claves del registro o tráfico
Gusano	Se replica por una red sin requerir ayuda directa del usuario	Puede degradar el rendimiento y propagarse por correo o protocolos estándar
Spyware	Recopila información del equipo y del usuario	Puede esconderse, cambiar el navegador o instalar barras de herramientas
Rogueware	Finge ser un antivirus o una herramienta útil	Muestra falsas infecciones e intenta cobrar o forzar instalaciones
Ransomware	Bloquea el equipo o cifra archivos para exigir un rescate	Su objetivo es el secuestro o la extorsión
Keylogger	Registra las pulsaciones del teclado	Puede ser software oculto o incluso hardware conectado al equipo

Conviene matizar algunos casos. En un **troyano** hay una acción visible y otra encubierta; en un **rootkit** son frecuentes las **puertas traseras**, que abren conexiones externas y escuchan en puertos concretos, y un ejemplo clásico es un **ps** troyanizado que oculta un proceso; un **gusano** suele componerse de un localizador de objetivos, un propagador, un mecanismo de control remoto, una interfaz de actualización, una rutina de *payload* y algún sistema de autorastreo. Los **gusanos de correo** son especialmente comunes.

Además de los tipos anteriores, también se citan:

- **Adware:** realiza acciones publicitarias ocultas, como falsos clics, para dar beneficio al atacante.
- **Hoaxes:** mensajes falsos sobre virus, amenazas o causas solidarias que buscan difusión masiva.
- **Exploits:** código diseñado para ejecutar acciones en otros sistemas, locales o remotos, aprovechando vulnerabilidades.

3.4.2 Objetivos del malware

Hoy en día, en un porcentaje muy alto de casos, la infección está ligada directa o indirectamente a un **móvil económico**.

El primer objetivo clásico es el **robo de información**: usuarios y contraseñas, historial de navegación, cookies, libretas de direcciones y también propiedad intelectual o industrial.

El segundo es el **secuestro**. Aquí entran el ransomware que bloquea el equipo y el que cifra sus ficheros; en ambos casos la condición para recuperar el uso del dispositivo o de los datos es **pagar**.

El tercero es el **reclutamiento para botnets**. Una **red de bots** o **zombies** es un conjunto de equipos infectados por un malware concreto y controlados desde un centro de **Comando y Control**. Un zombie aislado aporta poco; miles de ellos sirven para atacar otros equipos, enviar correo masivo, romper contraseñas, minar criptomonedas o robarlas. Cuantos más equipos infectados, mayor valor económico tiene la red, incluso como servicio alquilado a terceros.

3.4.3 Transmisión, propagación y activación

El malware puede llegar al sistema por muchas vías: programas de instalación, adjuntos, enlaces de descarga, fallos del navegador u otro software instalado, cracks o generadores de claves, dispositivos **USB** infectados, programas “gratuitos” descargados de sitios poco fiables y aplicaciones falsas que hacen más de lo que prometen.

Un truco de ingeniería social muy común es la **doble extensión** y la manipulación del icono del fichero, especialmente en Windows, para hacer pasar un ejecutable por un documento inocente.

Una vez presente en el sistema, el malware puede activarse de distintas formas: **ejecución única**, infección del **sector de arranque**, residencia en **memoria**, infección de **ficheros de aplicación**, infección de **librerías** o activación apoyada en **ingeniería social**.

3.4.4 Prevención y detección

Las medidas básicas siguen siendo muy pragmáticas: usar software de **fuentes fiables**, probarlo en **entornos aislados**, abrir solo adjuntos de los que se conoce su seguridad, tratar cualquier sitio web como potencialmente dañino y mantener **copias de seguridad**.

Los escáneres de malware buscan signos de infección usando **firmas** en disco y en memoria. Sus mecanismos de detección suelen basarse en **patrones de cadenas**, **patrones de ejecución** y

patrones de almacenamiento.

El problema es que el malware evoluciona rápido y los antivirus tradicionales no siempre consiguen mantenerse al día. Por eso muchos códigos maliciosos incorporan mecanismos de **autodefensa** para evitar ser detectados, dificultar el análisis, ocultarse en el sistema o entorpecer a antivirus y cortafuegos. El uso de **rootkits** forma parte de esa lógica.

3.4.5 Técnicas de evasión del malware

Tunneling

El **tunneling** crea un “túnel” entre el sistema operativo y el antivirus para proteger al malware de los módulos residentes que vigilan comportamientos típicos. Es especialmente frecuente en malware **residente en memoria**.

En sistemas donde el usuario trabaja con privilegios altos, el malware puede intentar **deshabilitar o matar** el antivirus o el cortafuegos, esquivar los mecanismos de detección por comportamiento, borrar campos de integridad de la base del antivirus, ejecutar su código “a través” del antivirus, troyanizar la base de firmas o impedir que el producto se conecte para **actualizarse**.

Armouring

Armouring agrupa técnicas destinadas a impedir que el malware sea analizado. El objetivo es dificultar el acceso a los ficheros abiertos, evitar el desensamblado o bloquear el *tracing* con depuradores.

Las técnicas citadas en los apuntes son: **antidesensamblador, cifrado de datos, ofuscación, compresión del código, antidepuración, antiheurísticas, antiemulación y anticebo**.

Virus polimórficos

Un **virus polimórfico** cambia su forma cada vez que infecta otro programa. La idea es eludir la detección por firma modificando instrucciones o incluso el algoritmo concreto que realiza la misma tarea. Un ejemplo típico es incluir varias rutinas de cifrado y descifrado y elegir una al azar en cada infección.

3.4.6 APT y MITRE ATT&CK

Una **APT** (*Amenaza Persistente Avanzada*) combina varias vulnerabilidades y técnicas de ataque para alcanzar objetivos a largo plazo. Puede afectar tanto a usuarios individuales como a entornos corporativos. En un escenario personal puede apoyarse en **perfiles falsos, ingeniería social**, aplicaciones de mensajería falsas o permisos abusivos, y terminar en extorsión o en venta de datos a terceros. En una organización puede continuar tras la primera infección mediante **escalada de privilegios y movimientos laterales**, con fines como robo de información, acceso no autorizado a recursos o secuestro de ficheros.

No todas las APT siguen exactamente las mismas fases, pero suelen cubrir desde el reconocimiento inicial hasta la explotación y la permanencia.

Para estudiar ese comportamiento se usa mucho **MITRE ATT&CK**, una base de conocimiento y taxonomía del comportamiento del adversario a lo largo del ciclo de vida del ataque. ATT&CK se divide en tres grandes entornos:

- **ATT&CK for Enterprise:** cubre redes y nube corporativas. Incluye Windows, macOS, Linux, PRE, Azure AD, Office 365, Google Workspace, SaaS, IaaS, redes y contenedores.
- **ATT&CK for Mobile:** se centra en **Android** e **iOS**.
- **ATT&CK for ICS:** se orienta a sistemas de control industrial.

En ATT&CK, las **tácticas** representan los objetivos del atacante en cada fase, y las **técnicas** o **subtécnicas** son las formas observadas de conseguirlos. De ahí que se hable a menudo de matrices de **TTPs** (*tactics, techniques and procedures*). Para implementar esas técnicas, el marco distingue entre **herramientas** de uso dual y **malware** específicamente malicioso. Dependiendo del objetivo del atacante y del sistema a comprometer, se usarán unas fases y unas técnicas u otras.

El valor práctico del marco es doble: permite estudiar campañas y grupos conocidos, y también consultar qué **mitigaciones** resultan útiles frente a determinadas **tácticas** o **técnicas**.

4. Seguridad en Redes

4.1 Introducción

La **seguridad en redes** busca proteger tanto la **información** como los **servicios de red** frente a accesos no autorizados, modificaciones, destrucción de datos, filtraciones de información confidencial y pérdida de **disponibilidad**.

4.1.1 Por qué las redes son vulnerables

Las comunicaciones pueden viajar por **cable**, **fibra óptica**, **microondas**, **WiFi** o **satélite**, y cada medio introduce debilidades propias. La idea de fondo es siempre la misma: el canal puede ser **interceptado**, **monitorizado** o **manipulado**. Por eso puede haber escuchas en enlaces cableados, receptores no autorizados en LANs inalámbricas o interceptación en enlaces por satélite y microondas.

4.1.2 Factores que aumentan la vulnerabilidad

Hay varios rasgos estructurales que hacen más difícil proteger una red:

- **Anonimato**: un atacante puede operar a distancia y dificultar su identificación.
- **Múltiples puntos de ataque**: cuantos más nodos y servicios existan, más superficies expuestas hay.
- **Compartición**: al conectar recursos, aumenta el número de posibles usuarios con acceso.
- **Complejidad**: conviven sistemas operativos, aplicaciones y dispositivos muy distintos.
- **Perímetro desconocido**: en redes grandes y cambiantes no siempre está claro qué forma parte de la red.
- **Ruta desconocida**: entre dos hosts puede haber varios caminos, algunos no confiables.

4.1.3 Debilidades de TCP/IP y ataques básicos de red

TCP/IP no se diseñó inicialmente con la seguridad como objetivo principal. Por eso, en su forma original, no incorpora mecanismos sólidos de **cifrado**, **autenticación** ni **integridad**. El resultado es que las comunicaciones pueden ser escuchadas, interceptadas o alteradas.

Modificación y fabricación de datos

Si la comunicación no está protegida, un atacante puede modificar o fabricar información. Las formas más habituales son:

- **corrupción** de datos;

- **secuenciación** indebida, cambiando el orden de llegada;
- **sustitución** de fragmentos por otros;
- **inserción** de nuevos valores;
- **repetición** de datos legítimos ya transmitidos.

Ataque de repetición

Un ataque de **repetición** o **replay** consiste en interceptar una comunicación válida y reenviarla después para que el receptor la acepte como legítima. La defensa básica es asociar a cada intercambio un identificador único, normalmente mediante **números de secuencia** o valores equivalentes, para que un mensaje viejo no pueda aceptarse otra vez.

Sniffing

El **sniffing** es la monitorización o interceptación no autorizada del tráfico de red. Puede ser:

- **pasivo**, si el atacante solo observa;
- **activo**, si además inyecta o modifica datos.

Las contramedidas más directas son el **cifrado**, la **segmentación de red**, los **IDS** para detectar sniffing activo y la monitorización de dispositivos no autorizados.

4.1.4 Reconocimiento y suplantación en red

Escaneo de puertos

El **escaneo de puertos** permite averiguar qué equipos están activos, qué servicios ofrecen y, a veces, qué sistema operativo o versión de software utilizan. Es una técnica útil para administración y auditoría, pero también para un atacante que busca objetivos fáciles. La herramienta más conocida es **Nmap**.

Con esta información un atacante puede identificar máquinas con un sistema operativo concreto, localizar servicios vulnerables o automatizar barridos de rangos completos de direcciones IP.

Reducir el riesgo pasa por tres medidas básicas: **no abrir más puertos de los necesarios**, **no exponer más información de la imprescindible** en los servicios y **mantener actualizado** el software.

Spoofing

El **spoofing** consiste en **hacerse pasar por otro elemento** dentro de la red. Puede darse en varias capas, por ejemplo como **IP spoofing**, **MAC spoofing** o **DNS spoofing**. El objetivo suele ser robar información, alterar comunicaciones o saltarse controles de acceso.

Las medidas mencionadas en los apuntes son la **autenticación basada en criptografía**, el **filtrado de tráfico**, las políticas de control de acceso, la monitorización y los **IDS**.

DNS spoofing

En el **DNS spoofing**, el atacante responde antes que el servidor legítimo y logra que la víctima asocie un dominio correcto con una IP falsa. En consecuencia, el usuario cree estar accediendo al sitio legítimo cuando en realidad se le redirige a un sistema controlado por el atacante.

ARP spoofing y Man in the Middle

ARP sirve para averiguar qué dirección **MAC** corresponde a una dirección **IP** dentro de una red local. Su debilidad es conocida: no autentica bien las respuestas, puede aceptar respuestas no solicitadas y cualquier equipo puede emitirlas.

Eso permite el **ARP spoofing** o **ARP poisoning**: el atacante envía respuestas ARP falsas, envenena la caché de las víctimas y consigue que el tráfico pase por él. De esta forma se sitúa entre dos extremos y ejecuta un **Man in the Middle (MitM)**, capturando, observando o modificando la comunicación mientras la reenvía para no levantar sospechas.

4.1.5 Protocolos inseguros y protocolos seguros

Protocolos inseguros

Algunos protocolos clásicos no incorporaban protección suficiente:

- **Telnet**: acceso remoto sin cifrado.
- **POP** en sus primeras versiones: credenciales y correo sin cifrar.
- **HTTP**: ni cifra ni autentica; si la información no es pública, la confidencialidad y la autenticación quedan expuestas.

Protocolos seguros

Los protocolos seguros añaden **confidencialidad**, **integridad** y **autenticación** usando técnicas criptográficas en distintas capas.

IPsec trabaja en la **capa de red**. Su objetivo es proteger paquetes IP de forma transparente a las aplicaciones. Entre sus piezas principales están:

- **ESP** (*Encapsulating Security Payload*), que protege la carga útil con autenticación, integridad y, normalmente, cifrado simétrico.
- **IKE** (*Internet Key Exchange*), que negocia claves y parámetros de seguridad usando criptografía asimétrica, Diffie-Hellman y, en muchos despliegues, certificados.

Su caso de uso típico es la **VPN**.

SSL/TLS opera en la **capa de transporte**. Aporta autenticación del servidor mediante certificados **X.509v3**, autenticación opcional del cliente, confidencialidad e integridad. Al iniciar una sesión, cliente y

servidor negocian una **suite criptográfica** que incluye, en esencia, mecanismo de autenticación, algoritmo de cifrado y función hash.

TLS se apoya en dos subprotocolos:

- **Handshake**, que autentica, negocia parámetros y establece la sesión segura.
- **Record**, que protege los datos ya intercambiados.

Históricamente, el *handshake* pudo usar **RSA**, **Diffie-Hellman fijo**, **Diffie-Hellman efímero** y mecanismos basados en **curvas elípticas**. El **Diffie-Hellman anónimo** no es recomendable porque queda expuesto a **MitM**. **TLS 1.3** simplifica y moderniza ese proceso.

SSH trabaja en la **capa de aplicación** y proporciona un canal cifrado y autenticado para administración remota. Sustituye a herramientas como **Telnet**, `rlogin` o `rsh`, y soporta autenticación por contraseña y por claves públicas.

Otros protocolos y tecnologías seguras citados en los apuntes son:

- **SFTP**, transferencia segura de ficheros sobre SSH.
- **DNSSEC**, que añade protección criptográfica a DNS y ayuda frente al DNS spoofing.
- **FTPS**, FTP protegido con TLS/SSL.
- **OpenVPN**, una VPN basada en TLS/SSL.

4.1.6 Ataques a la disponibilidad

En redes, la seguridad no trata solo de confidencialidad. También importa la **disponibilidad**: que el servicio siga funcionando. Puede perderse por problemas de enrutado, demanda excesiva o fallos de hardware y software.

DoS y DDoS

Los ataques de **denegación de servicio (DoS)** intentan impedir que un sistema siga prestando servicio. Los apuntes distinguen tres grandes familias:

- ataques **volumétricos**;
- ataques **basados en aplicación**;
- ataques de **deshabilitación de comunicaciones**.

Cuando intervienen muchas máquinas coordinadas, el ataque pasa a ser un **DDoS**. Suele apoyarse en una **botnet**, es decir, una red de equipos comprometidos controlados a través de una infraestructura de **Command and Control (C&C)**.

Ping of Death

El **Ping of Death** es un DoS clásico basado en el envío de paquetes **ICMP** malformados o demasiado grandes con el objetivo de provocar fallos en el sistema destino.

Smurf

El ataque **Smurf** falsifica como origen la IP de la víctima y envía peticiones **ICMP Echo** a una dirección de **broadcast**. El efecto es una amplificación: numerosos equipos responden a la víctima y la saturan.

SYN flooding

El **SYN flooding** explota el establecimiento de conexión en **TCP**. Cuando el servidor recibe un **SYN**, reserva recursos y queda a la espera del **ACK** final del cliente. Durante ese tiempo mantiene la conexión en estado **SYN_RECVD**.

Si el atacante envía muchas peticiones SYN con IPs falsificadas, el servidor responde con **SYN+ACK**, pero el ACK final nunca llega. Al llenarse la cola de conexiones pendientes, las peticiones legítimas empiezan a rechazarse.

El ataque se apoya en dos debilidades:

- la falta de autenticación fuerte del origen de los paquetes;
- la posibilidad de hacer **IP spoofing**.

Las contramedidas citadas se aplican en dos niveles:

Nivel	Medidas
Sistema	Reducir el <code>timeout</code> , aumentar la cola de peticiones pendientes y deshabilitar servicios no esenciales
Router	Bloquear paquetes externos con direcciones internas como origen y bloquear paquetes salientes con direcciones de origen que no pertenezcan a la red interna

Botnets

Una **botnet** es una red de equipos comprometidos controlados desde uno o varios nodos de **C&C**. Su valor está en la escala: permiten lanzar DDoS, distribuir malware, enviar correo masivo o apoyar otros ataques coordinados.

4.2 Cortafuegos

Un **cortafuegos** es el mecanismo que aplica una **política de control de accesos entre redes**. Puede ser hardware, software o una combinación de ambos, y su función es monitorizar y filtrar el tráfico que entra o sale de una red protegida.

En la práctica suele ser un dispositivo dedicado, porque así resulta más sencillo de analizar, mantener y optimizar. Su valor no está solo en bloquear paquetes, sino en convertir la política de seguridad en un conjunto claro de reglas.

Un cortafuegos bien diseñado debe cumplir tres propiedades: estar **siempre en el camino** del tráfico que pretende controlar, ser **resistente a manipulaciones no autorizadas** y mantenerse **lo bastante simple** como para poder ser analizado con rigor.

4.2.1 Qué puede y qué no puede hacer

Contra qué protege

Un cortafuegos protege frente a **accesos no autorizados** y **tráfico no permitido**, puede restringir también el tráfico **saliente** desde el interior y actúa como punto central para aplicar política de seguridad y auditoría. Naturalmente, también debe **protegerse a sí mismo**.

Limitaciones

No es una defensa total. Sus límites principales son:

- no evita **bugs** en aplicaciones o servicios ya permitidos;
- no protege frente a ataques que usan **conexiones autorizadas**;
- no resuelve por sí mismo la seguridad de la **red interna**;
- solo controla el tráfico que **realmente pasa por él**;
- puede resultar molesto para los usuarios si está mal diseñado;
- requiere una configuración y una administración cuidadosas.

Esto implica que el cortafuegos debe formar parte de una **política global**, no ser la única defensa.

4.2.2 Ejemplo de política

Una política típica puede expresarse como una secuencia de reglas. El ejemplo de los apuntes, pasado a tabla, permite correo SMTP y TFTP hacia la red interna, HTTP saliente desde la red interna y HTTP entrante hacia un servidor web concreto; el resto se bloquea:

Regla	Tipo	Origen	Destino	Puerto destino	Acción
1	TCP	*	192.168.1.*	25	Permitir
2	UDP	*	192.168.1.*	69	Permitir
3	TCP	192.168.1.*	*	80	Permitir
4	TCP	*	192.168.1.18	80	Permitir
5	TCP	*	192.168.1.*	*	Denegar
6	UDP	*	192.168.1.*	*	Denegar

4.2.3 Tipos de cortafuegos por tecnología

Filtrado de paquetes

Es la forma más básica: un router o dispositivo analiza cabeceras **IP**, **TCP**, **UDP** o **ICMP** y toma decisiones en función de una lista de reglas. Suele fijarse en:

- IP de **origen**;
- IP de **destino**;
- **protocolo**;
- **puertos**;
- **interfaz** por la que entra o sale el paquete.

Su comportamiento habitual sigue una de dos políticas:

- **denegar todo lo no permitido explícitamente**;
- **permitir todo lo no denegado explícitamente**.

Cada regla tiene una parte de **emparejamiento** y una **acción**. Las acciones típicas son:

- **aceptar**;
- **denegar** notificando al origen;
- **descartar** sin notificar;
- **registrar** o disparar acciones adicionales.

La gran ventaja del filtrado de paquetes es que es **rápido**, **escalable** e independiente de las aplicaciones. Sus límites son que la definición de reglas puede volverse compleja, que no entiende bien el **contexto** del tráfico y que muchas implementaciones confían demasiado en la dirección IP, que puede falsificarse.

Inspección con estado

El **filtrado dinámico** o **stateful inspection** no trata cada paquete como si estuviera aislado. Mantiene una **tabla de estados** con las conexiones activas, quién las inició y qué respuestas son coherentes. Así puede distinguir mejor entre petición y respuesta, reconocer el avance de una conexión TCP y permitir solo paquetes que encajan en una comunicación válida.

Además, muchos productos combinan esta idea con firmas o patrones sospechosos ya conocidos.

Cortafuegos de aplicación o proxy

En un cortafuegos de **capa de aplicación** no hay paso directo entre red externa e interna. Toda comunicación debe pasar por un **proxy**, de modo que existen dos conexiones separadas:

- una entre cliente y cortafuegos;
- otra entre cortafuegos y servidor.

El cortafuegos actúa como **servidor** frente al cliente y como **cliente** frente al servidor. Esto permite un control muy fino del protocolo y del contenido de cada servicio. Por eso suelen incorporar autenticación, auditoría detallada y restricciones más específicas que un filtro de paquetes.

Además, las reglas suelen ser **más fáciles de definir** que en un router con puro filtrado de paquetes, y cada proxy puede tratarse como un componente relativamente **independiente** dentro del sistema de defensa.

Ejemplos mencionados en los apuntes:

- un **proxy de correo** que escanea mensajes en busca de virus antes de reenviarlos;
- un **proxy FTP** que permite descargas pero bloquea el comando `put`.

A cambio, tienen peor **rendimiento**, rompen el modelo cliente-servidor directo y obligan a desplegar software específico por servicio.

Cortafuegos híbridos y NAT

La mayoría de los productos comerciales son **híbridos**: combinan filtrado de paquetes, inspección con estado y, cuando interesa, funciones de proxy. Algunos incluso se comportan como proxy en el establecimiento de la conexión y pasan a un tratamiento más ligero durante la transferencia de datos.

En este contexto suele aparecer **NAT**. No es un cortafuegos completo por sí mismo, pero añade opacidad: oculta la información TCP/IP de los hosts internos y hace que, desde fuera, toda la red parezca salir por una única IP pública. Eso dificulta que los equipos internos sean visibles directamente.

4.2.4 Arquitecturas de cortafuegos

Las configuraciones clásicas que aparecen en los apuntes son:

- **router de filtrado de paquetes**: un único router entre red interna y externa;
- **host de base dual** (*dual-homed bastion host*): un equipo intermedio que corta el tráfico directo entre ambas redes y sirve como bastión; en él se pueden instalar **proxies de aplicación** o permitir **login remoto** para acceder desde ahí al resto de hosts;
- **proxy con router de filtrado**: combina un bastion host con proxies y un router que solo permite el tráfico necesario hacia ese host.

4.2.5 DMZ

La **DMZ** es una subred intermedia entre **Internet** y la **intranet**. Su objetivo es alojar los servicios que deben ser visibles desde fuera sin exponer directamente la red interna. Allí suelen ubicarse servidores **web**, **correo**, **FTP** y **DNS**.

La idea es simple: desde Internet solo se debe poder alcanzar lo **estrictamente necesario**, y ese tráfico debe terminar primero en la DMZ. Como esos servidores están más expuestos, requieren

protección reforzada. En la DMZ también pueden convivir filtros, antivirus y zonas de cuarentena.

Como la DMZ está separada de la red interna, puede apoyarse también en funciones de **NAT** en el bastion host o en el cortafuegos para ocultar mejor la estructura interna.

DMZ para correo electrónico

En correo, el servidor de la DMZ actúa como filtro y ocultador de información interna, pero debe seguir siendo **transparente hacia el interior**. Cuando llega un mensaje desde Internet:

1. reensambla cabecera, cuerpo y adjuntos;
2. analiza contenido malicioso conocido;
3. revisa violaciones del protocolo SMTP;
4. comprueba las direcciones del destinatario;
5. reescribe la dirección hacia el servidor interno;
6. reenvía el mensaje.

En sentido saliente, además de lo anterior, analiza si hay **datos sensibles** y reescribe cabeceras que revelen nombres de host, correos o IPs internas, sustituyéndolos por el dominio del proxy o la IP externa del cortafuegos.

DMZ para servicios web

En tráfico **HTTP/HTTPS**, el cortafuegos o proxy puede inspeccionar las peticiones en busca de elementos sospechosos, por ejemplo líneas anormalmente largas o componentes maliciosos. Si detecta algo extraño, descarta la petición; si no, la reenvía al servidor de la DMZ.

4.2.6 Tipos de cortafuegos por ubicación

Los apuntes distinguen cuatro despliegues habituales:

Tipo	Uso típico	Rasgos
Personal	Un único equipo	Filtra entrada y salida, oculta el sistema frente a escaneos y conviene combinarlo con antivirus
SOHO	Pequeñas oficinas, típicamente de unos 2 a 50 usuarios	Suele ir integrado en equipos pequeños, delante del router o incluso dentro de él
Hardware	Oficinas medianas y sucursales	Gestión centralizada, funciones básicas y sistema propio del fabricante; ejemplos típicos: Cisco y Fortinet
Corporativo	Punto central de acceso a Internet en una empresa	Conecta varias redes, implanta la política global y suele ejecutarse sobre grandes servidores con configuraciones tolerantes a fallos

4.2.7 Configuración e implantación

Hay dos enfoques básicos:

- **diseño desde cero**, ajustado a necesidades concretas, pero más lento y costoso;
- **producto comercial**, con funcionalidades y plantillas ya disponibles.

Como referencia de trabajo, los apuntes citan las **guías del CCN-CERT**, en particular la **Serie 1000 de Guías de Procedimiento de Empleo Seguro**, buscando por “cortafuegos”.

Los errores más comunes al implantar un cortafuegos son:

- instalarlo sin una **política de seguridad** previa;
- añadir reglas por inercia, sin distinguir necesidades reales de deseos;
- pensar solo en el cortafuegos e ignorar otras medidas;
- no revisar **logs** ni **alarmas**;
- desactivar alarmas repetitivas que podrían ocultar ataques reales;
- permitir que demasiadas personas administren su configuración;
- dejar vías de acceso que **se salten** el cortafuegos.

Además, un cortafuegos suele apoyarse en otros controles:

- autenticación de usuarios externos mediante contraseña, certificados o tarjetas;
- antivirus, normalmente en la **DMZ**;
- filtros de contenido para **URL** y para controlar la **información saliente**;
- **IDS/IPS**;
- **VPNs** para acceso remoto seguro.

4.3 IDS/IPS

4.3.1 Intrusiones

Una **intrusión** es un intento de obtener acceso ilegal a un sistema o de realizar actividades no autorizadas sobre él. A partir de ahí conviene diferenciar:

- **detección de intrusiones**: descubrir accesos o actividades no autorizados;
- **prevención/protección**: actuar para evitar o limitar el daño.

4.3.2 Qué es un IDS

Un **IDS** (*Intrusion Detection System*) recoge información desde varias fuentes del sistema o de la red y la analiza según patrones de uso indebido o comportamientos anómalos. En algunos entornos puede integrarse con respuestas automáticas, pero su función básica es **detectar** y **avisar**.

Las tareas que aparecen en los apuntes incluyen monitorizar actividad de usuarios y sistemas, auditar configuración y vulnerabilidades, evaluar la integridad de sistemas críticos y ficheros de datos, identificar actividad anormal mediante análisis estadístico, revisar acciones administrativas que violen políticas y desplegar trampas o mecanismos equivalentes para observar al intruso.

4.3.3 IDS frente a IPS

Un sistema **IDS/IPS** busca detección y protección en tiempo real frente a ataques como **DoS**, **troyanos**, **escaneos de puertos** o **inyecciones**. Suele integrarse con:

- el **sistema operativo**, usando registros y auditorías;
- las **aplicaciones servidor**, que también deben generar logs útiles;
- el **cortafuegos**, que a menudo incorpora capacidades de detección o prevención.

La diferencia central es esta:

Sistema	Qué hace	Ubicación típica
IDS	Detecta, registra y genera alarmas, pero no corta el tráfico por sí mismo	Sonda fuera de línea, normalmente con una sola interfaz
IPS	Además de detectar, actúa para prevenir o bloquear	En línea, entre redes, normalmente con dos interfaces

Las respuestas posibles que citan los apuntes son reconfigurar el cortafuegos o ACLs de routers, emitir alarmas, guardar evidencias, lanzar programas de respuesta o terminar una sesión TCP.

4.3.4 Tipos de IDS/IPS por ubicación

Los tres despliegues básicos son:

- **software para servidores**, centrado en el host;
- **sondas de red**, que observan tráfico en una subred;
- **software para puestos de trabajo**, combinado a menudo con antivirus y cortafuegos personal.

NIDS

Un **NIDS** (*Network IDS*) usa sensores o agentes de red, normalmente basados en técnicas de sniffing.

Sus ventajas son que no altera datos existentes, puede detectar ataques a nivel de red como **SYN Flood**, permite vigilar redes grandes y trabaja en tiempo real.

Sus límites también son claros: puede sufrir en redes de muy alta velocidad, no puede inspeccionar bien contenido **cifrado** y no confirma si el ataque llegó a tener éxito; solo ve su rastro en el tráfico.

HIDS

Un **HIDS** (*Host IDS*) monitoriza eventos locales del sistema: logs del SO, logs de procesos y otros objetos que pueden no ser visibles para un NIDS. También puede vigilar el sistema operativo buscando **anomalías** y **firmas de ataques conocidos**. Tiene la ventaja de que puede observar el dato **antes de cifrarse** o **después de descifrarse**, así que funciona mejor en entornos con mucho tráfico cifrado.

Sus ventajas principales son:

- ver quién accede a qué;
- identificar mejor al usuario implicado;
- operar en entornos cifrados.

Sus inconvenientes son que no ve tan bien la actividad global de red, depende de la integridad del sistema operativo donde corre y consume recursos locales.

4.3.5 Cómo modelar una intrusión

Reconocimiento de firmas

El enfoque por **firmas** asume que una intrusión puede describirse mediante un patrón reconocible. El sistema compara los datos observados con una base de firmas conocidas, normalmente actualizada por el proveedor.

Su principal ventaja es el **bajo número de falsas alarmas**. Sus problemas son que las firmas deben codificarse a mano, que cuesta detectar variantes o ataques nuevos y que un atacante puede intentar imitar o confundir el patrón.

Los apuntes ponen varios ejemplos de señales útiles para un NIDS:

- **origen y destino** de los paquetes; por ejemplo, tráfico desde la **DMZ** hacia la red protegida o desde Internet hacia servidores que no ofrecen servicios directos al exterior, como una base de datos interna;
- **puertos** origen y destino;
- combinaciones sospechosas de **flags TCP**, por ejemplo **SYN** y **FIN** a la vez;
- el **campo de datos**, donde puede aparecer una petición sospechosa como **GET**
`../../../../etc/passwd HTTP/1.0`.

Detección de anomalías

La detección por **anomalías** compara la actividad observada con perfiles de comportamiento normal. Esos perfiles pueden construirse para **usuarios**, **grupos**, **aplicaciones** o **recursos** del sistema. Suele apoyarse en análisis estadístico y, en diseños más avanzados, en técnicas de IA como redes neuronales o minería de datos. Requiere fases de **aprendizaje** y conjuntos de entrenamiento amplios.

Su mayor fortaleza es que puede descubrir ataques **desconocidos** o patrones lentos y sofisticados. Su mayor debilidad es el riesgo elevado de **falsas alarmas**, sobre todo cuando cambian los hábitos normales de los usuarios o del sistema.

4.3.6 Cómo implantar un IDS

Los apuntes mencionan el **modo oculto**: una interfaz de red solo para recibir, sin dirección pública visible, y una segunda interfaz de control para enviar alertas o datos al servidor de gestión. La idea es que el IDS observe sin exponerse innecesariamente.

4.3.7 SIEM

Un **SIEM** (*Security Information and Event Management*) reúne información relevante de seguridad procedente de muchos productos hardware y software y la presenta en una consola unificada. Es una pieza habitual en un **SOC** (*Security Operations Center*), es decir, el equipo dedicado a monitorizar la red, detectar incidentes y coordinar su investigación y respuesta.

Los requisitos que aparecen en los apuntes son:

- portabilidad de datos;
- compatibilidad con distintas fuentes de logs;
- capacidad de personalización;
- almacenamiento;
- control de acceso y segregación;
- mantenimiento a tiempo completo.

Sus ventajas son la **centralización**, la **automatización**, el seguimiento de eventos para detectar anomalías, la consulta de históricos y la posibilidad de ver vulnerabilidades y explotación asociada. Sus desventajas son el **coste alto**, la **curva de aprendizaje larga** y la posible pérdida o limitación de control sobre la información generada.

4.4 Redes privadas virtuales

4.4.1 Introducción

Una **VPN** es una red que se extiende sobre una infraestructura pública mediante **encapsulación** y, habitualmente, **cifrado**. Los paquetes de la red privada viajan por un **túnel** a través de Internet u otra red pública.

Una VPN permite, por ejemplo, que un usuario remoto acceda a la red corporativa como si estuviera dentro de ella, con direcciones, privilegios y políticas equivalentes a los de esa red.

Los dos componentes básicos son:

- **dos terminales** de VPN, software o hardware, que encapsulan, autentican, cifran y descifran;
- **un túnel** entre ambos a través de la red pública.

Ventajas

La principal ventaja es que el cliente VPN se comporta como miembro de la red privada. Eso permite acceder a bases de datos, documentos internos y otros recursos corporativos, y además hace que las conexiones del cliente hacia Internet puedan salir usando los recursos y políticas de la red privada. También puede reducir costes de comunicación.

Inconvenientes

Las pegadas principales son la **carga adicional** sobre el cliente y el servidor, especialmente si se cifra; la mayor complejidad en el tráfico y en la configuración; y el hecho de que muchas aplicaciones ya cifran por sí mismas, así que una VPN no siempre añade seguridad real de extremo a extremo.

En ese último caso, conviene recordar una diferencia importante: si la aplicación cifra extremo a extremo, el cifrado protege todo el recorrido; en una VPN, el cifrado protege sobre todo el tramo entre el **cliente VPN** y el **servidor de túnel**.

4.4.2 Ejemplos de uso

Los escenarios más habituales son:

- **acceso remoto** a sistemas o información corporativa;
- **comunicaciones seguras entre oficinas** usando redes IP públicas;
- **teletrabajo**.

4.4.3 Tareas tecnológicas básicas

Toda VPN debe resolver tres tareas:

- **encapsulamiento IP**, envolviendo el tráfico original dentro de otro paquete;
- **cifrado**, para proteger la información transportada;
- **autenticación**, mediante servidores de autenticación o mecanismos de clave pública.

4.4.4 IPsec

IPsec trabaja en la **capa de red** y proporciona:

- **control de acceso**;
- **integridad**;
- **confidencialidad**;
- **autenticación del origen**, evitando suplantación de IP;

- **protección frente a repetición**, asegurando que cada paquete sea único en la sesión.

Los protocolos mencionados en los apuntes son:

- **ESP**, que protege la carga útil con autenticación, integridad y confidencialidad;
- **IKE**, que negocia claves y parámetros para que ESP pueda funcionar.

Modos de funcionamiento

IPsec puede operar de dos formas:

- **modo transporte**: protege solo la carga útil del datagrama IP, insertando la cabecera de IPsec entre la cabecera IP y los protocolos de capas superiores;
- **modo túnel**: encapsula el datagrama IP completo dentro de un nuevo paquete IP.

El **modo transporte** favorece la comunicación extremo a extremo, pero exige que ambos extremos entiendan IPsec. El **modo túnel** se usa mucho cuando el destino final no coincide con el dispositivo que realiza las funciones IPsec, como ocurre en muchas VPN de pasarela.

Encapsulación ESP

ESP añade autenticación, integridad y confidencialidad a los datos del paquete IP. En los apuntes se resume como cifrado de la carga útil, por ejemplo con **AES-GCM**, junto con un valor de autenticación/integridad. La cabecera IP exterior no queda protegida del mismo modo que la carga útil.

Los campos destacados en la estructura ESP son:

- **SPI** (*Security Parameter Index*): identifica la conexión o asociación de seguridad;
- **Sequence Number**: contador que aumenta con cada paquete y ayuda frente a repetición;
- **Payload Data**: datos protegidos;
- **Padding**: relleno para ajustarse al tamaño de bloque cuando hace falta;
- **Next Header**: indica el tipo de protocolo transportado;
- **Authentication Data**: contiene el valor de verificación de integridad.

IKE

IKE (*Internet Key Exchange*), apoyado históricamente en **ISAKMP** y **Oakley**, establece el contexto de seguridad entre dos nodos. En esencia realiza tres tareas:

1. negociar la política de seguridad, algoritmos y protocolos;
2. autenticar el intercambio **Diffie-Hellman**;
3. intercambiar y derivar las claves necesarias.

La **clave de sesión** resultante es esencial para que **ESP** pueda funcionar.

Integración con una PKI

Cuando hay muchos nodos que deben autenticarse entre sí, IPsec se beneficia de una **PKI**. Esta centraliza el alta y la baja de usuarios, simplifica la gestión de certificados y permite integrar tarjetas criptográficas, algo especialmente útil en escenarios de teletrabajo o movilidad.

4.4.5 Alternativas a IPsec

Las alternativas citadas en los apuntes son:

- **SSL VPNs:** VPNs en la capa de transporte; hoy se apoyan en **TLS**, y el ejemplo más usado es **OpenVPN**.
- **Túneles SSH:** permiten crear túneles entre puertos específicos para que una conexión local acceda a un recurso remoto o una conexión remota a uno local.

4.4.6 OpenVPN

OpenVPN usa **TLS** sobre un puerto configurable y puede trabajar sobre **TCP** o **UDP**. Soporta certificados, **PSKs** y usuario/contraseña para autenticación, y puede operar tanto a nivel de red como de transporte.

Los apuntes subrayan dos riesgos prácticos:

- el servidor puede enviar al cliente comandos a ejecutar, así que un servidor comprometido puede comprometer también a sus clientes;
- al ejecutarse como proceso de usuario, su **superficie de ataque** es mayor y ha sufrido vulnerabilidades en el pasado.

4.4.7 VPNs mediante túneles SSH

Los túneles **SSH** se usan a menudo en **bastion hosts** o equipos intermedios para saltar a otros sistemas. Herramientas como **OpenSSH** permiten crear interfaces de túnel entre host local y remoto y asociarles direcciones IP.

Sus ventajas prácticas existen, pero también sus costes:

- requieren instalar y configurar el cliente SSH en cada máquina;
- obligan a reconfigurar las aplicaciones que usarán el túnel;
- cada usuario necesita permisos de login en un servidor interno accesible desde Internet;
- suelen exigir conocimientos técnicos relativamente altos.

Por eso, el perfil de usuario más habitual de este tipo de VPN no es el usuario general, sino el **administrador de sistemas TI**.

cibertest_grupos_1_2_3_4

Cibertest - Bloque Criptografía

Grupo 4

1. El tamaño de la firma digital de un documento, ¿depende del tamaño de la función de resumen utilizada? Justifícalo.

O tamaño da firma dixital dun documento vén determinado polo tamaño da clave do algoritmo de clave pública escollido. Se usamos RSA de 2048 bits, a firma terá un tamaño de 2048 bits.

2. Los datos relativos a la seguridad de la conexión (algoritmos de cifrado y compartición de clave) en una conexión web https, ¿están asociados al certificado del servidor web? ¿De qué dependen?

No, no están asociados al certificados del servidor. Dependen de la negociación TLS, en concreto de la cipher suite acordada entre cliente (ClientHello) y el servidor (ServerHello.)

3. ¿Qué relación hay entre el tamaño de un fichero cifrado si ciframos con clave pública RSA de 1024 bits y si ciframos con clave pública RSA de 2048 bits en openssl?

En RSA, o tamaño do ficheiro cifrado resultante é sempre igual ao tamaño da clave utilizada. Os tamaños da clave dos dous algoritmos mencionados son 1024 e 2048 respectivamente, polo que o tamaño do ficheiro no segundo caso será o dobre do primeiro.

4. Si cifro un fichero de 16 bytes con AES-256-ECB, ¿qué tamaño tendría el fichero resultante? ¿Por qué?

O ficheiro resultante terá un tamaño de 32 bytes. Aínda que o tamaño do ficheiro coincide co tamaño de bloque en AES (128 bits ou 16 bytes), engádese por defecto un bloque completo de padding (PKCS#7)

5. Explica cómo sería el proceso para compartir un fichero de bckup de 200MB con otra persona de forma que se garantice su integridad, usando openssl

Se genera un hash del fichero con openssl dgst, se firma el hash también para garantizar autenticidad, y luego el receptor verifica el hash también con dgst y verifica la firma.

6. ¿Qué sucede si se cifra un bloque homogéneo (todos los caracteres iguales) en el modo de cifrado ECB de AES?

Para un único bloque homogéneo, no ocurre nada especial: AES en ECB lo cifra normalmente produciendo un único bloque de texto cifrado aparentemente aleatorio. El problema de ECB (patrones

visibles, bloques cifrados repetidos) solo se manifiesta cuando hay varios bloques idénticos.

7. ¿Tenemos que proporcionarle al receptor de un mensaje cifrado el vector de inicialización si usamos AES en modo CBC?

Si é necesario proporcionarlle o IV ao receptor. No caso de usar OpenSSL, este proceso é transparente, xa que o IV derívase do contrasinal e o salt (e o salt envíase no ficheiro cifrado). No caso de non usar salt en OpenSSL, o IV derívase exclusivamente a partir do contrasinal.

8. ¿Qué relación hay entre la función de derivación de clave utilizada para un algoritmo de cifrado simétrico y la contraseña utilizada por el usuario?

A función de derivación de clave (KDF) úsase para derivar a clave criptográfica que o algoritmo usa para cifrar. Esta clave derívase a partir do contrasinal e o salt. A KDF tamén soluciona o problema do tamaño da clave.

9. ¿Qué motivos pueden hacer que un certificado sea revocado?

Por compromiso de la clave privada, por compromiso de la CA, por cese de la actividad, por sustitución del certificado, por emisión incorrecta o por uso indebido.

10. Si cifro un fichero de texto con AES-128, y el fichero cifrado lo codifico en base64, ¿el resultado tendrá el mismo tamaño que el fichero cifrado? Explica la respuesta.

O ficheiro resultante en base64 será máis grande que o ficheiro cifrado orixinal (base 64 xera 4 bytes de texto ASCII por cada 3 bytes de datos binarios). Ademais, se o ficheiro cifrado non é múltiplo de 3, engádense caracteres de recheo ou padding (=) ao final para completar o último bloque de 4 caracteres.

Grupo 2

1. Si un navegador nos indica que la conexión es no segura porque el certificado de servidor está autofirmado, e ignoramos el aviso "aceptando los riesgos": ¿cuáles son los riesgos? ¿Qué dimensión de seguridad se ve afectada?

El riesgo principal es que el navegador no puede validar la identidad del servidor mediante una CA de confianza. Por tanto, un atacante podría suplantar la página o realizar un ataque de intermediario (MITM) y hacernos creer que estamos conectando con el servidor legítimo. La dimensión de seguridad afectada principalmente es la autenticidad, y como consecuencia también pueden verse comprometidas la confidencialidad e integridad de la comunicación.

2. Explica cómo sería el proceso para compartir un fichero de backup de 200MB con otra persona de forma que se garantice su origen (que es nuestro), usando openssl.

Para garantizar el origen hay que usar una firma digital. Primero se calcula un resumen del fichero, por ejemplo con `openssl dgst -sha256`, y después se firma ese resumen con la clave privada del emisor.

Se envía el fichero junto con la firma. El receptor calcula de nuevo el resumen del fichero recibido y verifica la firma con la clave pública del emisor. Si la verificación es correcta, se garantiza que el fichero procede del emisor y que no se ha modificado. No es necesario cifrar el fichero con AES/RSA si el enunciado solo pide origen/autenticidad y no confidencialidad.

3. ¿Para qué se usa el vector de inicialización en un cifrado AES en modo CBC? ¿Qué ocurre si es ECB?

En CBC, el IV se usa como valor inicial para el primer bloque: antes de cifrar, el primer bloque de texto plano se combina con el IV mediante XOR. Los bloques siguientes se combinan con el bloque cifrado anterior. El IV debe estar disponible para descifrar correctamente el primer bloque y normalmente debe ser aleatorio o impredecible, aunque no secreto. En ECB no se usa IV: cada bloque se cifra de forma independiente, por lo que bloques de texto plano iguales producen bloques cifrados iguales y pueden revelar patrones.

4. Si cifro un fichero de texto con AES-128, y el fichero cifrado lo codifico en base64, ¿qué ventaja puedo obtener?

La ventaja de codificar en base64 es convertir el fichero cifrado binario en texto ASCII, lo que facilita copiarlo, pegarlo, enviarlo por correo o transportarlo por sistemas que trabajan mejor con texto. No añade seguridad criptográfica: solo cambia la representación de los datos. Además, aumenta el tamaño aproximadamente un 33%, porque base64 representa cada 3 bytes binarios como 4 caracteres de texto, añadiendo de relleno si hace falta.

5. Si tengo el resumen SHA1 de un documento de texto, y añado 2 caracteres al texto, ¿a cuántos bits del nuevo resumen afectarán estos caracteres?

SHA1 siempre produce un resumen de 160 bits. Al añadir dos caracteres, por el efecto avalancha, el nuevo resumen será completamente distinto de forma impredecible: potencialmente pueden verse afectados todos los bits, aunque estadísticamente cambiará aproximadamente la mitad. Lo importante es que no solo cambian los bits correspondientes a esos caracteres, sino que el hash completo queda alterado.

6. ¿Qué relación existe entre la sal utilizada en un cifrado AES en modo CBC y el vector de inicialización?

La sal y el IV no son lo mismo. La sal se usa junto con la contraseña en la función de derivación de clave para obtener la clave de cifrado y, en OpenSSL, también el IV cuando se cifra con contraseña. La sal se guarda normalmente en la cabecera del fichero cifrado para que el receptor pueda derivar la misma clave y el mismo IV al descifrar. El IV no se almacena dentro de la clave; es un valor necesario para el modo CBC.

7. El tamaño de la firma digital de un documento, ¿depende del tamaño de la función de resumen utilizada? Justifícalo.

No depende normalmente del tamaño de la función de resumen, sino del algoritmo de firma y del tamaño de la clave. Por ejemplo, con RSA de 2048 bits, la firma ocupa 2048 bits, es decir, 256 bytes,

aunque se use SHA-256, SHA-512 u otra función hash. El hash influye en el valor firmado y en la seguridad, pero no determina por sí solo el tamaño final de la firma RSA.

8. ¿Qué contiene el certificado raíz de una autoridad certificadora?

Un certificado raíz contiene la identidad de la CA, su clave pública, el periodo de validez, número de serie, algoritmo de firma y extensiones como que es una CA (`CA:TRUE`) y los usos permitidos de la clave. Al ser raíz, normalmente está autofirmado: la propia CA firma su certificado con su clave privada. No contiene la clave privada de la CA; esa clave debe mantenerse secreta. Tampoco contiene los requisitos concretos de una petición de certificado, sino la información necesaria para que los clientes puedan confiar en los certificados emitidos por esa CA.

9. Cuando ciframos con AES-192-CBC un fichero de texto: ¿de qué depende el tamaño del fichero cifrado?

Depende del tamaño del fichero de entrada, del tamaño de bloque de AES y del relleno utilizado. AES siempre cifra en bloques de 16 bytes, también cuando la clave es AES-192. Si el fichero no es múltiplo de 16 bytes, se añade padding hasta completar el bloque. Si ya es múltiplo de 16 bytes y se usa padding PKCS#7, se añade un bloque completo adicional. Si se usa OpenSSL con sal, también puede aparecer una cabecera como `Salted__` más la sal, que añade bytes al fichero.

10. ¿De qué depende el tamaño de la clave generada por la función de derivación de clave en el cifrado simétrico en openssl?

Depende del algoritmo de cifrado elegido y del tamaño de clave que ese algoritmo requiere, no de la longitud de la contraseña introducida por el usuario. Por ejemplo, AES-128 necesita una clave de 128 bits, AES-192 una de 192 bits y AES-256 una de 256 bits. La contraseña, la sal, la KDF y las iteraciones determinan el valor de la clave derivada, pero el tamaño de la clave lo fija el cifrado seleccionado.

Grupo 1

1. ¿Qué contiene una CRL? ¿Quién debe firmarla?

Una CRL (Certificate Revocation List) contiene la lista de certificados que han sido revocados antes de su fecha de caducidad. Para cada certificado revocado suele incluir al menos su número de serie, la fecha de revocación y, opcionalmente, el motivo de la revocación. La CRL debe estar firmada por la autoridad certificadora (CA) que emitió esos certificados, o por una entidad autorizada por ella para firmar CRL. Así el cliente puede comprobar que la lista de revocación es auténtica y no ha sido modificada.

2. ¿Qué debe incluir obligatoriamente el certificado del servidor para que el navegador considere la conexión https segura?

Debe incluir una identidad válida para el servidor, normalmente en el campo Subject Alternative Name (SAN), que coincida con el nombre de dominio al que accede el navegador. Además, debe estar firmado por una CA de confianza para el navegador, estar dentro de su periodo de validez, no estar revocado y

permitir el uso correspondiente para autenticación de servidor. El certificado no contiene por sí solo todos los algoritmos de la conexión: esos se negocian durante el handshake TLS.

3. El tamaño de la firma digital de un documento, ¿depende del tamaño del documento? Justifícalo.

No. Antes de firmar se calcula un resumen o hash del documento, y lo que se firma es ese resumen. Por eso, aunque el documento sea muy grande, la firma no crece con el tamaño del documento. El tamaño de la firma depende principalmente del algoritmo de firma y del tamaño de la clave utilizada. Por ejemplo, con RSA de 2048 bits la firma tendrá 2048 bits, es decir, 256 bytes.

4. ¿Para qué se usa una función de derivación de clave en el cifrado simétrico en openssl?

Se usa para obtener la clave criptográfica que necesita el algoritmo de cifrado a partir de la contraseña introducida por el usuario. La contraseña no se utiliza directamente como clave AES: se combina normalmente con una sal y se procesa con una KDF para generar una clave binaria del tamaño adecuado, por ejemplo 128, 192 o 256 bits según se use AES-128, AES-192 o AES-256. En OpenSSL, cuando se cifra con contraseña, la KDF puede derivar también el IV necesario para el modo de cifrado.

5. Si tengo el resumen sha256 de un documento de texto, y añado 4 caracteres al texto, ¿a cuántos bits del nuevo resumen afectarían estos caracteres? ¿Por qué?

SHA-256 siempre produce un resumen de 256 bits. Al añadir 4 caracteres, por el efecto avalancha, el resumen resultante cambia de forma impredecible y potencialmente pueden verse afectados todos los bits del nuevo resumen. Estadísticamente se espera que cambie aproximadamente la mitad de los bits, pero la idea importante es que el cambio no afecta solo a una pequeña parte: el hash completo queda alterado.

6. ¿Qué tamaño debe tener el vector de inicialización en un método de cifrado AES-256 en modo CBC?

Debe tener el tamaño de bloque de AES, no el tamaño de la clave. AES tiene bloques de 128 bits, es decir, 16 bytes, tanto si se usa AES-128 como AES-192 o AES-256. Por tanto, en AES-256-CBC el IV debe ser de 16 bytes.

7. Cuando ciframos con AES-256 un fichero de texto: ¿de qué depende el tamaño del fichero cifrado?

Depende del tamaño del fichero original, del tamaño de bloque de AES y del padding utilizado. AES cifra en bloques de 16 bytes, aunque la clave sea de 256 bits. Si el fichero no ocupa un número exacto de bloques, se añade relleno hasta completar el último bloque. Si ya ocupa un número exacto de bloques y se usa padding PKCS#7, se añade un bloque completo adicional. Si se usa OpenSSL con sal, también puede añadirse una cabecera como `Salted__` más la sal.

8. Si cifro un fichero de texto con AES-128, y el cifrado lo codifico en base64, ¿qué sucede si lo abro en un editor de texto sin conocer la clave?

Veré texto ASCII imprimible, porque base64 convierte los datos binarios cifrados en caracteres de texto. Pero no podré leer el contenido original ni recuperar el mensaje sin la clave. Base64 no descifra ni

añade confidencialidad: solo cambia la representación del fichero cifrado para que pueda manejarse como texto.

9. ¿Qué sucede si se cifran bloques de texto idénticos en el modo de cifrado ECB de AES?

En ECB, cada bloque se cifra de forma independiente con la misma clave. Por eso, si hay bloques de texto plano idénticos, producirán bloques cifrados idénticos. Esto permite detectar patrones en el mensaje y es la razón por la que ECB no se considera un modo seguro para cifrar datos con estructura o varios bloques repetidos.

10. Explica cómo sería el proceso para compartir un fichero de backup de 200MB con otra persona de forma confidencial, usando openssl.

Para garantizar confidencialidad se debe cifrar el fichero. Como el backup es grande, lo adecuado es usar cifrado simétrico, por ejemplo AES-256-CBC o AES-256-GCM, con una clave o contraseña. Si se usa una contraseña compartida, OpenSSL deriva la clave mediante una KDF usando una sal y cifra el fichero con AES; el receptor, con la misma contraseña y la sal almacenada en el fichero cifrado, deriva la misma clave y descifra el backup. Si no se quiere compartir una contraseña directamente, se puede usar cifrado híbrido: cifrar el backup con AES y cifrar la clave AES con la clave pública del receptor. Así solo el receptor podrá recuperar la clave AES con su clave privada y descifrar el fichero.

Grupo 3

1. ¿Qué relación hay entre el tamaño de un fichero a cifrar y el tamaño de la clave pública si se usa RSA para el cifrado?

Con RSA no se puede cifrar directamente un fichero de tamaño arbitrario. El tamaño máximo del bloque que se puede cifrar está limitado por el tamaño de la clave RSA y, además, es algo menor que la clave porque hay que reservar espacio para el padding. Por ejemplo, con padding PKCS#1 v1.5 se reservan 11 bytes, así que con una clave RSA de 1024 bits (128 bytes) solo se pueden cifrar como máximo 117 bytes de datos. Para ficheros grandes se usa cifrado híbrido: el fichero se cifra con AES y RSA se usa para cifrar la clave AES.

2. ¿Cómo recupera el destinatario de un mensaje cifrado con AES-256 la clave utilizada a partir de la contraseña compartida?

El destinatario introduce la misma contraseña y aplica la misma función de derivación de clave que se usó al cifrar, con los mismos parámetros y la misma sal. La KDF deriva una clave de 256 bits, que es la que necesita AES-256. En OpenSSL, si el fichero fue cifrado con sal, la sal suele ir almacenada en el propio fichero cifrado, de modo que el receptor puede volver a derivar la misma clave a partir de la contraseña compartida.

3. Explica cómo sería el proceso para compartir un fichero de backup de 200MB con otra persona de forma que se garantice su origen (que es nuestro), usando openssl.

Para garantizar el origen se usa una firma digital. Primero se calcula un resumen del fichero, por ejemplo con SHA-256 usando `openssl dgst`. Después se firma ese resumen con la clave privada del emisor. Se envía al receptor el fichero junto con la firma, y el receptor verifica la firma usando la clave pública o el certificado del emisor. Si la verificación es correcta, sabe que el fichero procede del poseedor de la clave privada y que no ha sido modificado. No es necesario cifrar el backup si solo se pide garantizar el origen y no la confidencialidad.

4. ¿Qué contiene un certificado personal en formato PKCS#12?

Un fichero PKCS#12, normalmente con extensión `.p12` o `.pfx`, contiene el certificado personal del usuario y normalmente también su clave privada, protegida con una contraseña. Además, puede incluir certificados de la cadena de certificación, como certificados intermedios o el certificado de la CA, para poder validar el certificado personal. No debe confundirse con un certificado público simple, porque PKCS#12 suele usarse precisamente para transportar certificado y clave privada juntos de forma protegida.

5. Si cifro un fichero de 16 bytes con AES-128-CBC, ¿qué tamaño tendría el fichero resultante? ¿Por qué?

Tendría 32 bytes si se usa el padding habitual de OpenSSL/PKCS#7. AES cifra en bloques de 16 bytes. Aunque el fichero original ya ocupa exactamente un bloque, el padding añade un bloque completo adicional para indicar correctamente el relleno al descifrar. Por eso el resultado son dos bloques: 16 bytes de datos cifrados más 16 bytes correspondientes al bloque de padding.

6. El tamaño de la firma digital de un documento, ¿depende del tamaño del algoritmo de clave pública utilizado? Justifícalo.

Sí, normalmente depende del algoritmo de firma y del tamaño de la clave pública/privada utilizada. En RSA, por ejemplo, la firma tiene el tamaño del módulo RSA: con una clave de 2048 bits, la firma ocupa 2048 bits, es decir, 256 bytes. Lo que se firma suele ser el resumen del documento, pero el tamaño final de la firma no depende del tamaño del documento, sino de la clave y del algoritmo de firma.

7. ¿Qué sucede si se cifra un solo bloque homogéneo (todos los caracteres iguales) con AES en modo CBC?

Si solo hay un bloque, CBC lo combina mediante XOR con el IV antes de cifrarlo. Por tanto, aunque el bloque esté formado por caracteres iguales, el resultado será un bloque cifrado aparentemente aleatorio. Si se cifra de nuevo el mismo bloque con la misma clave pero con un IV distinto, el resultado será distinto. El uso del IV evita que un mismo primer bloque produzca siempre el mismo bloque cifrado.

8. ¿Cómo obtiene el receptor de un mensaje cifrado con AES-CBC el vector de inicialización necesario para descifrar?

El IV debe estar disponible para el receptor, pero no tiene que ser secreto. Puede enviarse o almacenarse junto al mensaje cifrado, normalmente al principio del fichero o del mensaje. En OpenSSL, cuando se cifra con contraseña y sal, el IV puede derivarse de la contraseña y la sal mediante la KDF; la

sal se almacena en el fichero cifrado, y el receptor la usa junto con la contraseña para obtener la misma clave y el mismo IV.

9. Si cifro un fichero de texto con AES-128, y el fichero cifrado lo codifico en base64, ¿añado confidencialidad? ¿Por qué?

No. La confidencialidad la proporciona el cifrado AES, no la codificación base64. Base64 solo transforma datos binarios en texto ASCII para facilitar su transporte o visualización en medios que trabajan con texto. Cualquiera puede decodificar base64 sin clave; lo que seguirá protegiendo el contenido será que los datos decodificados siguen estando cifrados con AES.